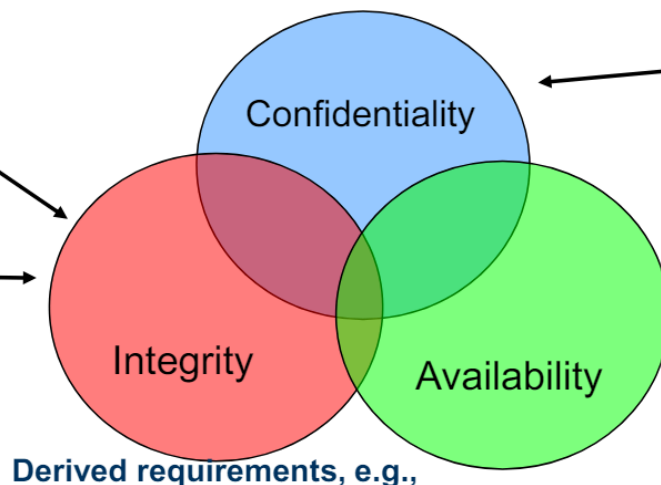


Security Goals (not only...)

Data integrity allows a receiver to verify that **data was not altered** by an adversary during its transmission

Data freshness ensures that messages have **not been replayed**.



Sensitive information must be **protected from disclosure** to unauthorized parties

Derived requirements, e.g.,

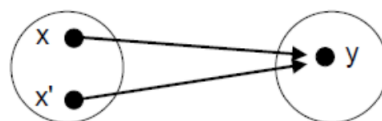
Authentication: Who is who?

Access control: Only selective access is authorized

Eigenschaft/Modi	ECB	CBC	CTR (beste)
Parallelisierbar(gut)	Ja	Nein	Ja
Deterministisch(schlecht)	Ja	Nein	Nein



One-way property



[Strong] Collision resistance := Hash-Eigenschaften

- **MAC:** (Hash mit Schlüssel) Integrität und Authentizität. Eine Nachricht wird mit geheimem Schlüssel K gemacced. Nachricht und Mac werden öffentlich gesendet. Der Empfänger löst den Mac mit geheimem Schlüssel K und die enthaltene Nachricht, wenn das Ergebnis stimmt, ist Integrität und Autorität gescheckt.

- **Sicherheitsziele der MAC:**

- **Selective Forgery (S. Fälschung):** Angreifer kann ein neues Nachrichten-MAC-Paar für eine Nachricht seiner Wahl zu erstellen.

- **Existential Forgery (E Fälschung):** Der Angreifer kann ein neues Nachrichten-MAC-Paar erzeugen, hat jedoch keine Kontrolle über den Inhalt der Nachricht.

- HMAC... Hash mit MAC verknüpft: $HMAC(K, m) = H((K' \oplus opad) || H((K' \oplus ipad) || m))$.

- Public Krypto Dank: Faktorisierung: RSA, Diskrete Logarithmus: DH-Schlüsselaustausch, DSA, ElGamal, ECDSA

- ECDSA > RSA (läuft schneller)

Confusion: verschleiert die Beziehung zwischen dem Schlüssel und dem Chiffretext.

Diffusion: vermischt die Beziehung zwischen dem Klartext und dem Chiffretext.

Example of Some "Hard" problems

Multiplication:

- Given primes p and q , compute $N = p \cdot q$.
- This is very easy to compute, since we just multiply p and q .
- The inverse problem: given N find p and q is called **factoring**.

Modular exponentiation:

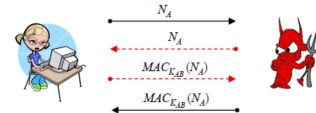
- Given N and an element $a \in \mathbb{Z}_N$, compute $b \equiv a^m \pmod{N}$.
- This can be computed efficiently using repeated squaring.
- The inverse problem: given N , $a, b \in \mathbb{Z}_N$ find m such that $b \equiv a^m \pmod{N}$ is called the **discrete logarithm** problem.

Lecture 2: Protocol Security Datei

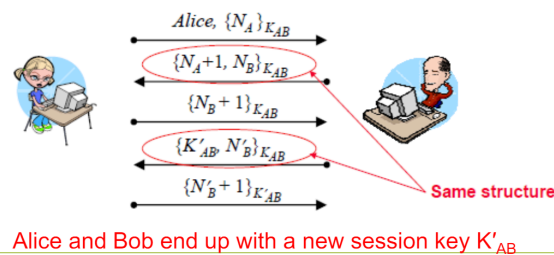
Nonce Freshness: Ein Angriff anhand vorheriger Informationen kann nicht stattfinden. Soll gemacced werden und nicht verschlüsselt: Integrität ist wichtig, nicht Sicherheit.

Parallel Session Attack: Alice enthält eine K_{pr} , der Bob gehören sollte, aber kann nicht unterscheiden, wessen K sie enthält. **Lösung**: Die ID von Bob fließt auch ein. Identität soll verschlüsselt übertragen werden.

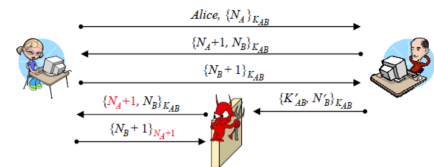
Reflection Attack: Nonce niemals als Session-Schlüssel, ein Nonce ist nicht sicher genug dafür.



Gleiche Struktur bietet Angriffsmöglichkeit.



If nonces and keys have the same length (128 bits) then Eve can replay the message



Alice would accept N_A+1 as the new key

Challenge-Response Authentication (handshake)

- Eine Partei sendet Challenge, die andere beantwortet mit pre-agreed Infos, welche Freshness haben.
- Challenge und Response kann mit Nonces und Timestamps erzeugt werden.

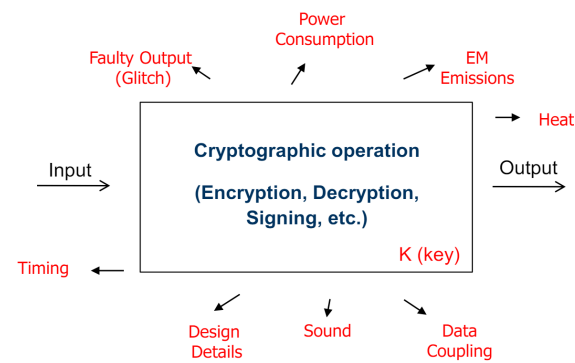
Wichtig: Wenn in einem Protokoll verschiedene Nachrichten, verschiedene 'Ziele' wie Schlüssel Agreement und Authentication verwendet werden, dann mische sie durch: Bspw: ID im Key-Agreement nutzen.

- Signal von einem Autoschlüssel kann empfangen und weitergeleitet werden: Dadurch können Autos gestohlen werden.

Lecture 3: Side Channel Attacks Datei

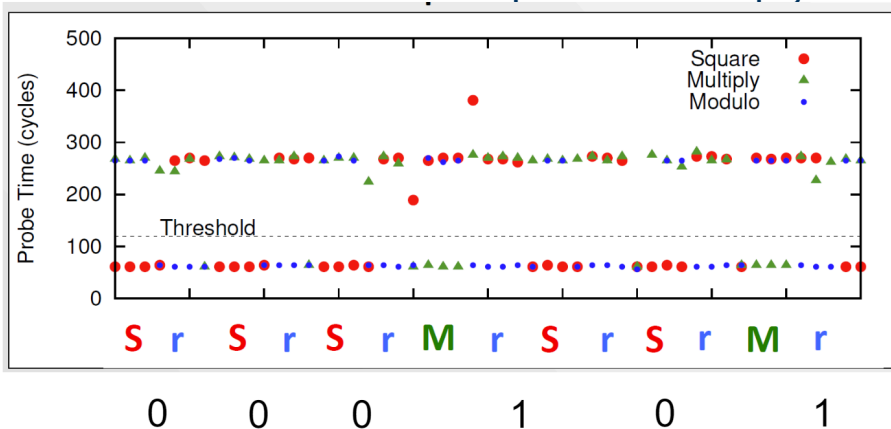
Side channel analysis: Versuch durch Messung unterschiedlicher Werte sicherheitsrelevante Informationen zu sammeln, um bspw. Schlüssel zu rekonstruieren.

- RSA nutzt Multiply-Square-Algorithmus, um schnell exponentieren zu können. Algorithmus wurde wie folgt implementiert: (x ist geheime Schlüssel)
 - Langsam: if k -te-Bit Stelle von $x = 1$ then Multiplikation
 - Schnell: if k -te-Bit Stelle von $x = 0$ then nur $R_k = S_k$
- Durch Ausführungszeit der Operationen private Schlüssel finden.
- Lösungsvorschläge:
 - 1: Fake Multiplikation bei $x=0$
 - 2: (echt) Masking message vor Entschlüsselung



Cache-Timing-Attack: Flush & Reload Attacks: geheime Informationen durch Überwachung von Speicherzugriffen zu erhalten. Beim Square & Multiply-Algorithmus beobachtet der Angreifer den Cache, um herauszufinden, welche Speicherbereiche bei der Berechnung verwendet werden. Durch gezieltes Leeren und Neuladen des Caches kann der Angreifer ableiten, welche Teile des geheimen Schlüssels gerade verwendet werden. Dies ermöglicht es ihm letztendlich, den gesamten Schlüssel zu extrahieren.

Flush & Reload Attacks on Square & Multiply



If $x_b=1$ then

Multiply -> Modulo reduce ->

Square -> Modulo reduce

Otherwise

Square -> Modulo reduce

- $x_b = 1$: MrSr, $x_b = 0$: Sr. Treshhold'un altında kalan neyse o an o işlem yapılyordur.

Maßnahmen:

Die kryptographischen Operationen **immer in der gleichen Reihenfolge** ausgeführt werden, wodurch die Entschlüsselung unabhängig vom geheimen Schlüssel läuft.

- Schreibe Software, die in konstanter Zeit arbeitet.

Spezifische Gegenmaßnahmen gegen Cache-Timing-Angriffe:

Vermeide Speicherzugriffe

Speicher-Maskierung: das Zugriffsmuster auf den Speicher verschleiern

Verstecken des Timings

Deaktiviere Cache

Cache-based side channels on AES: Keine Cache-Nutzung für AES

Meltdown und Spectre: Wenn man x Instruktion benutzt hat, wird man vermutlich zunächst y Instruktion nutzen (basiert auf prediction: Prefetching) - Deaktivierung der Cache (only seen possible solution)

Physical Problems: Eye-Retina-Reflection -> Kamera Access to see retina to read password. Aber auch Tastatur-Tracking usw.

Lecture 4: Password & Authentication Schemes Datei

Warum Authentifikation? Ermöglicht Kontosperrung nach mehreren Versuchen: Verhindert Brute-Force-Angriffe.

How to Auth? Wissen (Geburtsdatum), Besitz (Credit Card), Biometrik (FaceID, Retina), Does (Signature)

Multi Factor Authentication, at least, two-factor authentication -> zwei verschiedene Geräte müssen aktiv werden.

Log in mit Passwort 2 Wege der Hashing: (one-way-eigenschaft von Hashwerte)

Client-Side-Hashing: Passwort Eingabe-> Hashwert von Passwort berechnet durch JavaScript auf eigenen Prozessor ->

Hash wird versendet->'Amazon' vergleicht Hashwerte. Nachteil: Replay Angriffe (Hash erneut nutzbar).

Server-Side-Hashing (Beliebt): Passwort Eingabe-> Passwort wird verschlüsselt (durch HTTPS) und versendet-> Amazon
hasht das Passwort und vergleicht Hashwerte.

Problem: Schwache Passwörter anfällig gegen Wörterbuchangriffe.

Lösung: Salt hinzufügen (öffentliche Parameter), damit der Angreifer sein Wörterbuch speziell für diesen Angriff basteln muss, keine vollständige Verteidigung. "Vollständige" Verteidigung gegen Rainbow-Attacks (vorberechnetes Wörterbuch; gegen neue Salt nicht nutzbar).

Problem: Hash-Berechnung sehr schnell.

Lösung: Memory-Bound-Hash: Mehrmals hashen: Brute-Force schwieriger.

Zugriff auf Passwortdateien: • make the etc/passwd file unreadable to unprivileged users.

• etc/passwd world-readable (all), but hashed passwords stored in a separate file – etc/shadow

Challenge-Response: Authentication durch: Use ZK protocol to allow Alice to demonstrate knowledge of a secret without revealing the secret (or any other useful information) to Bob.

One-Time-Protocol: Lamport's Hash: Ablauf:

- Alice sends a username.

- Server replies with a integer n.

- Alice selects a password but never sends it to server, instead she hashes it n-Mal und speichert alle Zwischenergebnisse
 $x \rightarrow h(x) \rightarrow h^2(x) \rightarrow \dots \rightarrow h^n(x)$.

- Alice sends $h^n(x)$ back to server.

- Next time she wants to log in, Alice sends $h^{n-1}(x)$ to Server, Server prüft ob $h(h^{n-1}(x)) = h^n(x)$ übereinstimmt. Nächstes Mal sendet sie h^{n-2} usw.

- Adding salt to password vor erstem Hash allow Alice to login to different servers with the same password.

- Nachteile: Server muss man vertrauen -> Man-In-The-Middle -> Eve wählt n klein und Wörterbuchangriff darauf.

Strong password protocols: Eavesdropper will not obtain enough information to do good password guesses.

Encrypted Key Exchange (DH): N_A -> Nonce von Alice, N_S -> Nonce von Server, $W=h(x)$, $K=g^{ab}$

SPEKE (%99 wie DH) uses W in place of g in the DH exchange.

Problem der EKE und SPEKE: Server muss W speichern.

Augmented Strong Password Protocols: Server muss W nicht speichern + wir müssen Server nicht vertrauen.

- Server weiß p und $2^W \bmod p$, kann aber daraus W nicht berechnen, wegen Discrete-Log-Problem

- Secure Remote Password: bringt zusätzlich die Eigenschaft: Neue a und b Wahl -> Neue K: Fast wie one time Passwort.

Access Control: Entity to owner/group/world. Wer hat welche Zugriffe/Operationen auf welche Objekten.

DAC: Nutzer -> Welche Objekte -> Welche Operationen

Role-Based Access Control: Nutzer -> Rolle -> Welche Objekte -> Welche Operationen : Rechte für Rollen wichtig.

Linux Security Model (Android nutzt auch linux)

Linux treats everything as a file expect users & groups: **giriş/çıkış işlemleri** özel dosyalar üzerinden yapılır, z.B: **/dev/usb**

- Owner (User) – Group – World (Others) r:read = 4, w:write = 2, x:execute = 1, -:izin yok = 0.

- Kernel-Mode -> Zugriff auf alles (root = uid 0) Schutz des Zugriffs auf Root ist sehr wichtig.

- Auch wenn der Angreifer root wird, kann er nur Hashwerte der Passwörter sehen, aber sie nicht verändern.

- Password files not writable (not even root), how does changing password work?

▪ -r-s--x--x 1 root sys 15688 Oct 25 1995 /bin/passwd

▪ -r--r--r-- 1 root sys 7627 Oct 1 12:50 /etc/passwd

▪ -r----- 1 root sys 3292 Oct 2 15:14 /etc/shadow

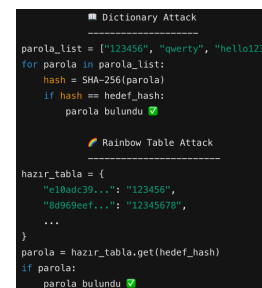
Sicherheit: Genaue Auswahl der Rechte jeweiligen Apps: Wetter App hat Zugriff auf Internet, Wallet App (worin

Credit-Card liegt) hat keinen Zugriff auf Internet. Sie können kommunizieren durch bspw. Helligkeitsänderung sei 1 und

keine -änderung 0: dadurch codiert ein App "geheime Informationen" wie ein Credit-Card-Passwort, die Wetter-App liest

es und versendet es im Internet. Die Kommunikation zwischen zwei Apps kann man nicht verhindern. Einzige

"theoretische Lösung" ein App per Gerät.



SOFTWARE SECURITY

Lecture 5: Password & Authentication Schemes Datei

Evil input: Command oder Script oder Code: kann SQL Injection, XSS, Buffer Overflows erzeugen.

Maßnahme: Input Validation: Whitelisting(with Regex best way), Blacklisting, Length Check, Regular Expressions usw.

Input Validation auf Client nicht anvertrauen und auf Server Input Validation auch ausführen.

- **Syntactic validation:** correctness structured fields: Geburtstag: Tag, Monat, Jahr.

- **Semantic (logical) validation:** correctness of values in a specific context: start date is before end date.

SQL ist eine standardisierte Programmiersprache, um Datenbanken zu erstellen, zu verwalten und zu manipulieren.

SQL-Injection ist eine Angriff auf Datenbankanwendungen, bei dem böartigen SQL-Code in eine Abfrage eingefügt wird, um unbefugten Zugriff auf Daten zu erhalten oder diese zu manipulieren. Datenabfrage mit SELECT

Beispiel: the user enters a number, this generates an http get request for: https://www.pizza.com/show_orders?month=10

normaler SQL-Abfrage:

```
String sql_query = "SELECT pizza, quantity, order_day "
+ "FROM orders "
+ "WHERE userid=" + session.getCurrentUserId()
+ " AND order_month=" + request.getParameter("month");
```

normaler Antwort sieht bspw so aus:

```
SELECT pizza, quantity, order_day
FROM orders
WHERE userid=4123
AND order_month=10
```

- Attacker makes a modified http request: https://www.pizza.com/show_orders?month=0%20OR%201%3D1

- '%20' steht für ein Leerzeichen. '%3D' für ein das Gleichheitszeichen '=' also insgesamt: '0 OR 1=1'

request.getParameter("month") returns '0 OR 1=1': Eve gets every entry in the table, including private data of other users.

Input als String Maßnahme:

```
+ " AND topping= \"
+ request.getParameter("topping") + \"
```

- but easy to circumvent, create a search for
abc'; DROP TABLE creditcards; --

Input validation strategies

RE: Beispiel: `^[A-Z]{2,4}?[1-9][0-9]{1,3}$`

^: Start des Inputs.

[A-Z]{2,4}?: Zwei bis vier Großbuchstaben (optional, durch das ?). Es

kann also auch übersprungen werden.

[1-9]: Eine Ziffer von 1 bis 9.

[0-9]{1,3}: Ein bis drei Ziffern (0 bis 9).

\$: Ende des Inputs.

Keine Buffer wie (links) `/^(a+)+$/` oder `/^(a+)*$/` erstellen: Input: "aaa...a". CPU binlerce

kombinasyonu boşuna deniyor. Bu şekilde CPU'yu meşgul edip servisi çökertirsin.

Maßnahmen spezifisch gegen SQL-Injection

Prepared Statements: Erlauben die Erstellung von Abfragen mit Bind-Variablen.

Bind-Variablen: Platzhalter für Parameter, die garantiert keine Steuerzeichen haben.

- Die Eingabe wird immer als String interpretiert und somit niemals ausgeführt.

Zugriff auf verschiedene Ordner konfigurieren: Access control wie rechts

- Verschlüsse alle wichtige Daten und Schlüssel in gut geschützte Ordner reinlegen.

- Disable unused services

- Prevent leaks of database schema: don't display error messages/stack traces to users

- ALLE diese Punkte sollten gemeinsam agieren, um Systemsicherheit aufzubauen.

The Morris Worm (Buffer Overflow Attack) Ziel: Uzak sistemde root erişimi almak

- Zielobjekt: fingerd servisi (Port 79) Kullanıcı bilgilerini gösteren basit bir ağ servisi

Wenn eine Verbindung zum fingerd hergestellt wird, liest dieser Daten aus dem Netzwerk—unter Verwendung von gets()

fingerd hatte 512 Byte großen Puffer im Stapelspeicher (ama uzunluk kontrolü yoktu)

- Angriff: Morris sent 537 byte → (536 bytes data + 1 newline = which made gets() stop reading input)

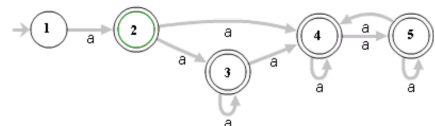
Overflowing the buffer and overwriting the return address → saldırganın kodu çalıştırıldı

The fingerd program ran as root, so when it was attacked to run a shell, that shell also ran as root.

- The constructed SQL query is

```
SELECT pizza, quantity, order_day
FROM orders
WHERE userid=4123
AND toppings='abc';
DROP TABLE creditcards ; --'
```

- '--' represents an SQL comment, thus ignores all following characters such as the unmatched '



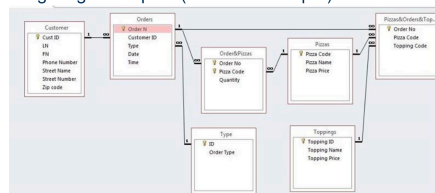
```
db.prepareStatement(
    "SELECT pizza, quantity, order_day "
    + "FROM orders WHERE userid=? "
    + "AND order_month=?");
```

```
(1, session.getCurrentUserId());
(2,
    Integer.parseInt(request.getParameter("month")));
res = ps.executeQuery();
```

Bind variable
data
placeholder

parameters
typed

Mitigating the impact (Defense in Depth)



♦ Payload-Inhalt:

Başta 400 NOP (no operation) ((komutu → return address → NOP sled → shellcode olması için. Düşün ki shellcode senin hedefin ama nişan almak zor. O yüzden onun önüne bir "kayma pisti" (NOP sled) kuruyorsun. Sen sadece bu pistin herhangi bir noktasına (örneğin ortasına) ateş ediyorsun. Kurşun kayarak hedefe ulaşıyor.))

Dann wurde Shell-Befehl (/bin/sh) ausgeführt → saldırgana root shell açtı

Heartbleed (keep-connection-alive)

Normale Nutzung Heartbleed-Extension: Heartbeat(5,ABCDE) -> Server sendet ABCDE zurück.

Angreifer nutzen aus: Heartbeat(64.000,ABCDE)-> Server sendet ABCDE,ms1,ms2... zurück. (Alles was er hat(64.000 größte Speicherplatz bei Server hier nur angenommen)). -> Die zusätzlichen 63.995 Byte können kritische Informationen zu Schlüsseln enthalten.

Wie kann man die enthaltenen Daten identifizieren, ob es Schlüssel ist oder nicht? Schlüsseln sind typisch lang.

🔧 Kısa Yorum / Ders:

gets() gibi güvenli olmayan (uzunluk kontrolü olmayan) fonksiyonlar buffer overflow'a yol açar

Bellek taşması ile kod çalıştırmak mümkündür

Gegenmaßnahmen:

- Längenüberprüfung (Die Sprache C hat es nicht)

- - - BURAYA KADAR KISALTTIM - - -

Lecture 6: X86 INTRO

Verständnis:

- Unterliegende Systemarchitektur verstehen: memory access, data storage, code execution, ...
- Analyse von Programmen: Wenn etwas in Binary liegt, das in eine verständliche Code umwandeln.
- Memory Access (um folgende Angriffe zu verstehen): buffer overflow attacks, return-to-libc attacks, ...
- Intel x86 architecture and low-level assembly programming.

Code Generation

- Source Code wird vom Compiler in Binary (Maschinencode) übersetzt, damit er ausführbar ist.
- Binary Code: 11001010 01100110 ⇔ Machine language: mov al, 61h

Assembler: Niedrigste Ebene, mit der wir arbeiten können. Übersetzt fast direkt in Maschinencode

Beispiel: Intel x86 Assembler (32-bit)

Intel x86 (32-Bit) Architektur: 32-Bit-System → max. 4 GiB RAM ($2^{32} = 4.294.967.296$ byte-adressierbare Speicherstellen)

CISC-Architektur: Wenige universelle Register, viele spezialisierte Register. Viele komplexe Befehle (z. B. direkter AES-Call). Komplexes Speichermanagement

Intel x86 – Register: Register = kleine, extrem schnelle Speichereinheiten direkt in der CPU.

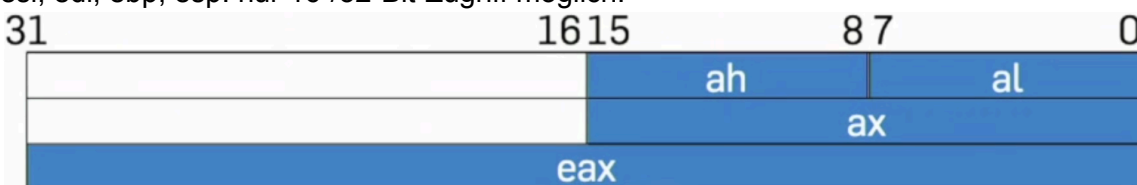
Dienen zur Zwischenspeicherung von Daten und zur Ausführung von Befehlen. für: Rechnen, Adressierung, Funktionsaufrufe, Stack-Operationen.

In Assembler wie Variablen ansprechbar.

Register	Zweck
eax	Akkumulator – Hauptregister für arithmetische/logische Operationen, speichert oft Rückgabewert
ebx	Basisregister – Hält Basisadressen für Speicherzugriffe
ecx	Zähler – Wird oft als Schleifenzähler verwendet
edx	I/O-Register – Dient bei Division & Multiplikation, auch für I/O
esi	Quelle bei Stringoperationen (Source Index)
edi	Ziel bei Stringoperationen (Destination Index)
ebp	Base Pointer – Zeigt auf Beginn des aktuellen Stackframes
esp	Stack Pointer – Zeigt auf die aktuelle Spitze des Stacks

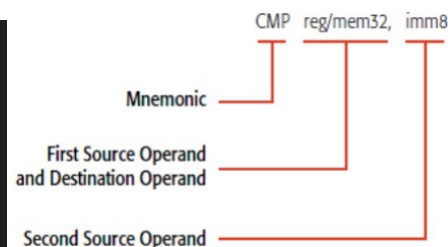
📌 Hinweis: Registerverwendung ist konventionell, keine harte Regel.

Registeradressierung: Register sind 32 Bit lang. Zugriff auf Teile eines Registers möglich (z. B. `eax` → `ax` → `al` / `ah`)
`esi`, `edi`, `ebp`, `esp`: nur 16-/32-Bit-Zugriff möglich:



Befehlsaufbau (Intel-Syntax)

- Format: `mnemonic ziel, quelle` → z.B. `sub eax, ecx` bedeutet `eax = eax - ecx`
- Hex: `h` am Ende (`10h` = 16)
- Speicherzugriffe in eckigen Klammern: `mov eax, [ebx]`



Befehle

Arithmetic operations: <code>add</code> , <code>sub</code> , <code>mul</code> , <code>rol</code> , <code>shl</code> , ...		Comparison/jumps: <code>cmp</code> , <code>jmp</code> , <code>jXX</code>	
<code>add eax, ecx</code>	<code>eax = eax + ecx</code>	<code>cmp eax, ecx</code>	Compare content of <code>eax</code> and <code>ecx</code> and set <code>eflags</code> accordingly
<code>sub edx, ebx</code>	<code>edx = edx - ebx</code>	<code>jmp 2342</code>	Set <code>eip</code> to 2342
<code>mul ecx</code>	<code>eax = eax * ecx</code>	<code>jz 1234</code>	Set <code>eip</code> to 1234 if <code>ZF</code> (Zero Flag) is set in <code>eflags</code> register (which is the case when the two operands of the previous comparison are equal)
<code>div ebx</code>	<code>eax = eax / ebx</code> , <code>edx = eax % ebx</code>	...	
<code>rol eax, 12</code>	Rotate <code>eax</code> 12 bit to the left		
<code>shl eax, 1</code>	Shift left (=multiply <code>eax</code> by 2)		
Data transfer: <code>mov</code>			
<code>mov eax, ecx</code>	<code>eax = ecx</code>		
<code>mov edx, [1234]</code>	<code>edx</code> = content of memory address 1234		

Division erklärt (div ebx):

- `eax` = 10 (Dividend)
 - `ebx` = 3 (Divisor)
 - Der Befehl `div ebx` führt die Division `eax / ebx` aus.
- Ergebnis:**
- **Quotient:** `10 / 3 = 3` (wird in `eax` gespeichert)
 - **Rest:** `10 % 3 = 1` (wird in `edx` gespeichert)

shl eax: Shift Left (Linksverschiebung) **Erklärung:**

- **Funktion:** Der Befehl `shl eax, 1` verschiebt alle Bits im `eax`-Register um eine Position nach links. Das Bit, das ganz links herausfällt, wird verworfen, und die leere Position ganz rechts wird mit einer 0 gefüllt.
- **Mathematischer Effekt:** Multiplikation des Wertes in `eax` mit 2.

Beispiel:

Angenommen, `eax` enthält den Wert 3.

- **Binärdarstellung vor der Verschiebung:**

`eax = 3` (binär: 0000 0000 0000 0000 0000 0000 0000 0011)

- **Nach `shl eax, 1`:**

Die Bits werden um eine Position nach links verschoben:

- Binär: 0000 0000 0000 0000 0000 0000 0000 0110
- Dezimal: `eax = 6`

- **Ergebnis:**

Der Wert in `eax` hat sich durch die Linksverschiebung von 3 auf 6 verdoppelt.

(`shr eax, 1` - Shift Right (Rechtsverschiebung) **analog**)

`rol eax, n` verschiebt die Bits im `eax`-Register um `n` Positionen nach links und rotiert die herausgeschobenen Bits zurück an die rechte Seite.

Dieser Befehl wird häufig in Algorithmen verwendet, bei denen die Reihenfolge der Bits relevant ist, wie etwa in bestimmten Verschlüsselungsmechanismen.

`cmp eax, ecx` wenn `eax` und `ecx` identisch, dann 0 Flag

`jmp 2342` führt einen sofortigen Sprung zu einer bestimmten Adresse aus.

`jz 1234` springt nur dann zu einer bestimmten Adresse, wenn das Ergebnis einer Operation null ist (d. h., wenn zwei verglichene Werte gleich sind) (If-Conditioning)

`jg` springt, wenn der erste Operand größer als der zweite ist.

`jae` der erste Operand größer oder gleich der zweite. "Jump if Above or Equal"

Beispiel:

Angenommen:

- `eax = 10`
- `ecx = 5`
- 1. **`cmp eax, ecx`:**
 - Vergleicht `eax` mit `ecx`.
 - Da `eax` (10) größer als `ecx` (5) ist, bleibt das Carry Flag (CF) auf 0.
- 2. **`jae 1234`:**
 - Da `CF = 0` ist, wird der Sprung zu Adresse 1234 ausgeführt.

Vor `inc eax`: `eax = 5` **Nach `inc eax`:** `eax = 6`

`byte ptr`: Bezieht sich auf einen 8-Bit-Wert (ein Byte).

`word ptr`: Bezieht sich auf einen 16-Bit-Wert (ein Wort).

`dword ptr`: Bezieht sich auf einen 32-Bit-Wert (ein Doppelwort).

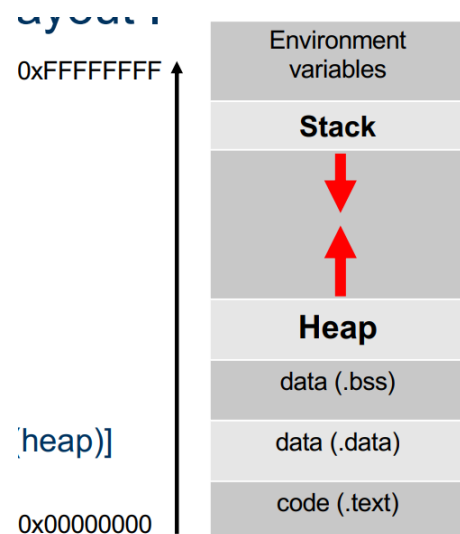
`qword ptr`: Bezieht sich auf einen 64-Bit-Wert (ein Quadwort).

Speicher

- linearer, kontinuierlicher Adressraum
- kleinste adressierbare Einheit: 1 Byte
- Maximal können 4 GiB adressiert werden.

Speicherlayout (Bereiche=Segmente)

- Stack-Segment: Lokale Variablen und Informationen zur Ablaufsteuerung (z.B. Rücksprungadressen bei Funktionsaufrufen) gespeichert.
- Daten-Segment: Umfasst globale uninitialisierte (.bss) und initialisierte (.data) Variablen sowie dynamische Variablen (Heap).
- Code (Text) Segment: Enthält Programmcode, der schreibgeschützt ist.
- Stack startet in hohen Adressen und geht runter zu niedrigen, Heap andersrum!



Endianness

Es gibt zwei verschiedene Arten, Speicher zu adressieren: little-endian, big-endian

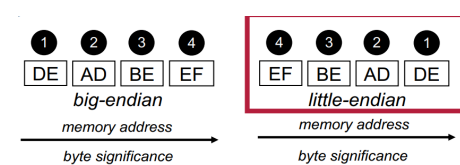
- Beschreibt die Art und Weise, wie Speicher gelesen/geschrieben wird.

Beispiel: Wert im Speicher: DE AD BE EF (4 Byte)

Most significant Byte: in big-endian in höchste memory address.

in little-endian in niedrigste memory address.

Wir verwenden (x86-Architektur) das Little-Endian-Format.



- Bei Speicher Manipulieren, müssen wir auf die Endianness der Architektur achten! → Buffer Overflows

Speicheradressierung

- Speicherzugriffe in der Assemblersprache werden in [eckigen Klammern] dargestellt.
- Eine Instruktion kann höchstens einen Speicherzugriff ausführen.
- Es gibt vier verschiedene Arten von Speicherzugriffen (nicht nur auf mov beschränkt):

Instruction	Type	Explanation
mov eax, 1337	Immediate	Load constant into eax
mov eax, ebx	Register	Load ebx into eax
mov eax, [2342]	Memory	Load content from static memory address
mov esi, [edi]	Offset	Load content from memory address via pointer in register

Stack und Funktionsaufrufe

Typischerweise beinhaltet die Ausführung eines Programms verschiedene verschachtelte Funktionsaufrufe, bei denen Argumente an die aufgerufene Funktion übergeben werden.

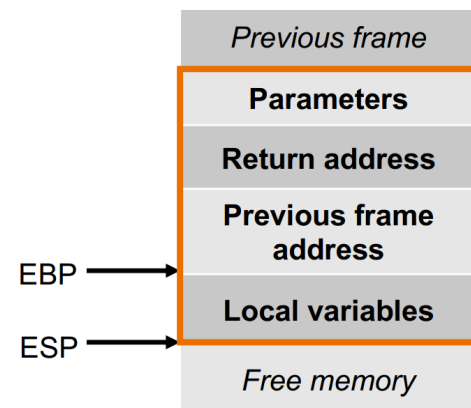
Wie wird das gemacht?

Der Stack ist ein Speicherbereich, der für lokale Variablen und Informationen zu Unterprogrammaufrufen reserviert ist.

```
int foo(int var1, int *bar2) {  
    [...]  
    tmp = bar(23, var1);  
    [...]  
}
```

Jeder Funktionsaufruf hat ein Stack-Frame:

- Eingabe- und Ausgabewerte (Stack-Parameter)
- Rücksprungsadresse
- Gespeicherter Registerinhalt (alter EBP)
- Lokale Variablen
- Der Base-Pointer (**ebp**) ist ein Register, das auf den Anfang des aktuellen Stack-Frames zeigt. Dies erleichtert den Zugriff auf die Stack-Parameter und die lokalen Variablen innerhalb der Funktion. Anstatt die genaue Position der Daten im Stack suchen zu müssen, kann der Prozessor einfach den Base-Pointer und einen Offset verwenden, um die gewünschten Werte schnell zu finden.



Stack-Ausrichtung:

Abbild 1

- **esp** muss durch 4 teilbar sein.
- Es können nur Vielfache von 4 Bytes (32 Bit) auf den Stack gepusht und vom Stack gepoppt werden.
- Die Adressierung relativ zu **ebp** erfolgt daher ebenfalls in Vielfachen von 4.

Stack-Operationen: push, pop: (Beide von Spitze heraus bzw. von ganz unten nach **Abbild 1**)

- **push ecx:** Verringert **esp** (um 4) und kopiert dann den Quelloperanden **ecx** an die Spitze des Stacks.
Erklärung: Wenn ein Wert auf den Stack geschoben wird, geschieht dies in zwei Schritten. Zuerst wird der Stack-Pointer (**esp**) um 4 verringert, um Platz für das neue Element zu schaffen. Anschließend wird der Wert aus dem Register **ecx** auf diesen neuen Platz im Stack geschrieben.
- **pop esi:** Kopiert das oberste Element des Stacks nach **esi** und erhöht **esp** um 4.
Erklärung: Wenn ein Wert vom Stack gezogen wird, geschieht dies ebenfalls in zwei Schritten. Zuerst wird der Wert vom obersten Element des Stacks in das Register **esi** kopiert. Dann wird der Stack-Pointer (**esp**) um 4 erhöht, um anzuzeigen, dass das Element vom Stack entfernt wurde.

Subroutines (Unterprogramme)

Bisher haben wir gelernt:

- Ein neues Stack-Frame wird bei jedem Funktionsaufruf auf den Stack gelegt.
- Es speichert alle Informationen, die für diesen Funktionsaufruf relevant sind.- Nach der Rückkehr aus der Funktion können wir diese Informationen nutzen, um den Zustand vor dem Funktionsaufruf wiederherzustellen und die Programmausführung fortzusetzen (das aktuelle Frame wird gepoppt).

Subroutines: call, ret

call 2342	Call function at address 2341, push address of next instruction to stack (return address saved in frame)
ret	Return from subroutine, use the instruction pushed previously onto the stack (pop it from stack)

call 2342: Ruft die Funktion an der Adresse 2342 auf und schiebt die Adresse der nächsten Anweisung auf den Stack (die Rücksprungadresse wird im Frame gespeichert).

Erklärung: Die **call**-Anweisung ruft eine Funktion auf, indem sie die Adresse der nächsten Anweisung (die nach dem Funktionsaufruf ausgeführt werden soll) auf den Stack schiebt und dann zur angegebenen Funktionsadresse springt. Dadurch kann das Programm nach Beendigung der Funktion wieder an der richtigen Stelle fortgesetzt werden.

ret: Rückkehr aus dem Unterprogramm, wobei die zuvor auf den Stack geschobene Anweisung verwendet wird (sie wird vom Stack gepoppt).

Erklärung: Die **ret**-Anweisung beendet die aktuelle Funktion und kehrt zur Adresse zurück, die durch den **call**-Befehl auf den Stack gelegt wurde. Diese Adresse wird vom Stack gepoppt (entfernt), und die Programmausführung wird dort fortgesetzt.

Funktionsparameter werden auf den Stack gelegt, bevor die Funktion mit der push-Anweisung aufgerufen wird:

- Der letzte Parameter wird zuerst gepusht.

Prolog / Epilog

10h: 10 Hexadezimal

[...]: Execution of Function f

Der **Prolog** (Setup) bereitet den Stack für den Funktionsaufruf vor. Er richtet das Stack-Frame ein, speichert wichtige Register (wie den Base-Pointer **EBP**), und bereitet den Speicherplatz für lokale Variablen vor.

```
push ebp      ; Speichern des aktuellen Base-Pointers
mov ebp, esp  ; Setzen des neuen Base-Pointers
sub esp, 8    ; Speicher für lokale Variablen reservieren (z.B. 8 Bytes)
```

Der **Epilog** (Cleanup) wiederherstellt den Stack und die Register, bevor die Funktion zurückkehrt. Er stellt den ursprünglichen Zustand des Stacks und der Register wieder her, wie sie vor dem Funktionsaufruf waren.

```
mov esp, ebp  ; Stack-Pointer auf Base-Pointer zurücksetzen
pop ebp       ; Ursprünglichen Base-Pointer wiederherstellen
ret           ; Rückkehr zur aufrufenden Funktion
```

Stack-Frame + Assembly (+High-level Language)

Return address -> call function

old ebp -> push ebp -> wir wollen es speichern, weil wir alte ebp-pointer (bei 4)

speichern wollen. Denn es wird gleich verändert.

esp/ebp auf gleiche Stelle -> mov ebp, esp.

esp eine Stelle runter -> sub esp, 4. Damit wir Platz für neue Variablen haben.

ebp - 4 (letzter Platz der Stack, den wir mit sub esp, 4 erstellt haben)

- Damit haben wir die Funktion durchgeführt und jetzt geht es mit

Epilog weiter.

pop ebp: Der Wert old ebp wird vom Stack in das ebp-Register

geladen. esp wird um 4 Bytes erhöht, sodass es nun auf die

Rücksprungadresse zeigt. Ebp-Zeiger ganz oben.

ret 8: Die CPU nimmt die Rücksprungadresse vom Stack und springt

zur entsprechenden Adresse zurück, um das Programm fortzusetzen.

Danach wird esp um 8 Bytes erhöht, um die beiden Parameter (4 und y) vom Stack zu entfernen.

```
mov [var_x], eax ; RET
```

return Adress in eax; RET, das wird in var_x reinkopiert.

Intel x86 – Aufrufkonventionen

Beim Umgang mit Funktionsaufrufen muss ein Compiler die folgenden drei Probleme lösen:

f(1, 2, 3)

```
push 3
push 2
push 1
call f
```

Function f

```
push ebp      ; save old ebp
mov ebp, esp  ; load ebp with esp
sub esp, 10h  ; reserve space for
               ; local variables
[...]
mov esp, ebp  ; load esp with ebp
pop ebp       ; restore ebp
ret           ; return
```

High-level language

```
x = func1(y, 4);
stdcall int func1(int a, int b)
{
    int z;
    z = a * b;
    return z;
}
```

ebp →

y

return address

old ebp

esp →

Stack frame

Assembler

```
push 4
push [var_y]
call func1

func1:
    push ebp
    mov ebp, esp
```

High-level language

```
x = func1(y, 4);
stdcall int func1(int a, int b)
{
    int z;
    z = a * b;
    return z;
}
```

ebp + 0xC

ebp + 0x8

ebp + 0x4

ebp

ebp - 0x4

z = y * 4

Stack frame

ebp

esp

Assembler

```
push 4
push [var_y]
call func1

func1:
    push ebp
    mov ebp, esp
    sub esp, 4
    mov ecx, [ebp+0x8]
    mul ecx, [ebp+0xC]
    mov [ebp-0x4], ecx
    mov eax, [ebp-0x4]
```

Assembler

```
push 4
push [var_y]
call func1
mov [var_x], eax ;RET

func1:
    push ebp
    mov ebp, esp
    sub esp, 4
    mov ecx, [ebp+0x8]
    mul ecx, [ebp+0xC]
    mov [ebp-0x4], ecx
    mov eax, [ebp-0x4]
    mov esp, ebp
    pop ebp
    ret 8
```

1. Wie erfolgt die Übergabe von Parametern?
2. Wann wird der Stack-Pointer während eines Funktionsaufrufs verändert?
3. Welche Register müssen während eines Funktionsaufrufs gespeichert werden?

Eine Aufrufkonvention bestimmt, wie diese Probleme gehandhabt werden, und muss innerhalb eines Programms – insbesondere bei der Verwendung externer Bibliotheken – konsistent sein.

Für uns sind drei relevante Aufrufkonventionen von Bedeutung: **cdecl**, **stdcall**, und **fastcall**.

cdecl	stdcall	fastcall
- All parameters are pushed to stack Calling function cleans the stack after the function call returns - C semantic <i>Advantage: variable number of parameters (e.g., printf)</i> - ecx and edx are available for use in the function - Return value available in eax	- All parameters are pushed to stack Callee is responsible to cleanup the stack - ecx and edx are available for use in the function - Return value available in eax - Standard calling convention for Win32 API	- First two parameters in ecx and edx - All other parameters on stack - Return value available in eax - Typically only used internally by OS

cdecl	stdcall	fastcall
Call: <pre> push 3 push 2 push 1 call F add esp, 12 cmp eax, 0 jz error </pre> Subroutine F: <pre> push ebp mov ebp, esp ... pop ebp ret </pre>	Call: <pre> push 3 push 2 push 1 call F cmp eax, 0 jz error </pre> Subroutine F: <pre> push ebp mov ebp, esp ... pop ebp ret 12 </pre>	Call: <pre> push 3 mov edx, 2 mov ecx, 1 call F cmp eax, 0 jz error </pre> Subroutine F: <pre> push ebp mov ebp, esp ... pop ebp ret 4 </pre>

Unterschiede:

- Bei fastcall werden nur 2 Variablen in eax und ecx gemovt, der Rest in Stack. Darum am Ende 4 Byte als Return
- Bei cdecl add esp, 12 denn 12 Byte für 3 Variablen, die gereturnt werden. Am Ende keine Byte Angabe.
- Am Ende werden alle Variablen gereturnt (12 Byte).

Registersicherung (Register Preservation)

- Der aktuelle Ausführungszustand, also der Inhalt der Register, muss gespeichert werden, bevor eine Funktion aufgerufen wird, da diese auf den Registern arbeiten könnte...
- ... und muss wiederhergestellt werden, wenn die Funktion beendet wird.
- Zuerst werden alle Register auf den Stack gepusht, und dann in umgekehrter Reihenfolge wieder vom Stack gepoppt!

Lecture 7: SW VULNERABILITIES

Privilege Escalation: Ein Angreifer erlangt unberechtigte Rechte (z. B. Root-Rechte) auf einem System.

Race Condition – TOC/TOU-Angriff:

Ein typischer Angriff nutzt die Zeit zwischen

TOC (Time of Check): Prüfung der Berechtigungen

TOU (Time of Use): tatsächlicher Zugriff

Angriff:

1. Run program: Input 1: filename, Input 2: string: "root:myhashpassword".
2. TOC => validated
 - 2.1 Interrupt to execute another program.
 - 2.2 This program replaces "filename" by a soft link to /etc/passwd, da UID 0 ist, wird /etc/passwd geöffnet.
3. Schließlich write "root: mypasswordhash" into file /etc/passwd.

Integer Over-/Underflows

Begrenzter Wertebereich von Ganzzahlen führt bei Überschreiten zu Über- oder Unterläufen (z. B. 8-Bit: $255 + 1 = 0$).

Buffer Overflow entsteht, wenn mehr Daten in einen Puffer geschrieben werden als Platz ist (z. B. 8 Byte Buffer, 10 Byte Daten). Angriff auf Stack oder Heap möglich.

Stack: Lokale Variablen, wächst von oben nach unten (LIFO).

Heap: Dynamischer Speicher, wächst von unten nach oben.

Beispiele:

```
int len = 805
|buf| = 805 dann 5 Byte Overflow
```

```
int len input: -1
[1] len < sizeof(kbuf)
[2] len > 800 : da hier len unsigned int wird!
```

- buf[256] und i fängt an bei 0 hochzuzählen, also i sollte höchstens 255 sein können.
- korrekte Version: $i \leq 255$.

Struktur eines Stacks:

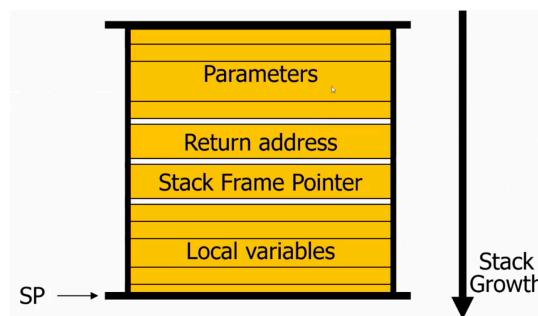
Für jeden Funktionsaufruf wird ein Stack-Frame zum Stack hinzugefügt.

Ein Stack-Frame speichert alle Informationen einer Aufgabe, die notwendig sind, um diese später fortzusetzen.

Beispiel: Reihenfolge der Funktionsaufrufe: f_1, f_2, f_3

Struktur eines Stack-Frames (von oben nach unten):

- Parameter
- Rücksprungsadresse
- Stack-Frame-Pointer (Basiszeiger EBP)
- Variablen



Stack-Overflow

Rücksprungsadresse überschreiben yaparak beliebige

Konsolenausgaben yapabilirsin: Overflow with pointer to shell:

exec("/bin/sh"): dann shell-access.

Access to a file

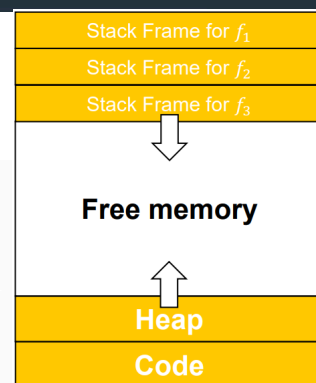
```
if (access("filename", OK) == 0)
{ fp = fopen("filename", "w"); }
```

Overflows	- unsigned ints, $> 2^{32}-1$ - signed ints, $> 2^{31}-1$ - floats, $> 3.40282347e+38$ - doubles, $> 1.7976931348623157e+308$
Underflows	- unsigned ints < 0 - signed ints, $< -2^{31}$ - floats, $> 1.17549435e-38$ - doubles, $> 2.2250738585072014e-308$

```
int copy_something(char *buf, int len)
{
    char kbuf[800];
    return memcpy(kbuf, buf, len);
}

int copy_something(char *buf, int len){
    char kbuf[800];
    if(len > sizeof(kbuf)){ /* [1] */
        return -1;
    }
    return memcpy(kbuf, buf, len); /* [2] */
}
```

```
#include <stdio.h>
void func(char *str)
{ char buf[256];
  int i;
  for (i=0;i<=256;i++) buf[i] = str[i];
}
int main(int argc, char **argv)
{
  func(argv[1]);
}
```



When the function is invoked the stack looks like:



What if `*str` is 136 bytes long? After `strcpy`:



Format String Attacks

“%n” Attack

- `printf(“%n”, &x)` will change the value of the variable x

“%s...%s” Attack

>

Einige unsichere C-Bibliotheksfunktionen:

- `strcpy(char *dest, const char *src)`
- `strcat(char *dest, const char *src)`
- `gets(char *s)`
- `scanf(const char *format, ...)`
- `sprintf(const char *format, ...)`

```
int func(char *user) {  
    fprintf( stdout, user);  
}
```

Problem: what if `user = “%s%s%s%s%s%s%s%s”` ??

- Most likely program will crash: DoS.
- If not, program will print memory contents. Privacy?

Gegenmaßnahmen zu unterschiedlichen Zeitpunkten:

Bevor wir überhaupt mit dem Programmieren beginnen

Beim Testen

Beim Ausführen des Codes

Während der Entwicklung

Don't use C or C++ (use type-safe languages) Typ-sichere Sprachen (z.B. Java, Meta Language).

- If you use, low risk use: use safe Befehle statt unsichere!

Vermeidung riskanter Programmierkonstrukte:

- `fgets` > `gets`.
- `strn*` > `str*`.
- `snprintf` > `sprintf` & `vsprintf`.
- Überprüfen Sie immer Sonderfälle: Negative Werte. Große Werte. Länge der Zeichenketten.

Statische Quellcodeanalyse ist eine Methode zur automatischen Überprüfung von Quellcode auf Sicherheitslücken wie Buffer Overflows. Tools wie Coverity helfen dabei, viele, aber nicht alle Fehler zu finden.

Stack-Overflow-Countermeasures (not against heap overflow):

Randomisierung: (Zufällige Veränderung interner Systemdetails): Gegen: **Return-Oriented Programming**: Ausführen von beliebigem Code ohne das Injizieren von neuem Code.

ASLR: Zufällige Festlegung von Stack-Startpunkts, von Heap-Startpunkts, von Variablen.

Comprehensive Randomization: Vieles mehr wird randomisiert, vgl. zur ASLR.

Nicht-ausführbarer Stack: injected code on stack wird nicht ausgeführt.

return to libc-attack: Return adresini değiştirerek: put a point to the already existing code like `system()`. Dann auf stack "sh" Fonksiyon bittikten sonra program, eigentliche return adresse değil, `system("/bin/sh")` fonksiyonuna gider. Shell açılır.

Overflow-Countermeasures:

Canaries:



To corrupt random canary, attacker must learn the pseudorandom string.

Idee: Angreifer muss auch canary overflowen; überprüfe ob canary verändert wurde. canary wert ist geheim & dynamisch.

Shadow Stack: Speichere lokale copy bzw. shadow copy von dem Stack. Bevor Ausführung vergleiche return address: wenn return address verändert wurde: nicht ausführen. - Shadow Stacks ist nicht erreichbar von Nutzers address space.

Laufzeitüberprüfung: StackGuard, Libsafe, SafeC, (Purify).

Libsafe schützt Programme vor Buffer Overruns, durch well-made Längenüberprüfung.

Libverify erweitert diesen Schutz, indem es die Rücksprungadresse überwacht und auf Manipulationen überprüft.

Lecture 8: VULNERABILITY DETECTION & MALWARE

Formal verification

Formal nachweisen, dass ein Software Stück eine Menge von Anforderungen erfüllt. Wird verwendet, um das Vertrauen in ein System zu erhöhen.

Idee:

Erstellen eines formalen Modells von:

- Der Systemimplementierung (was die Software leistet)
- Den Systemanforderungen (was die Software leisten muss)

Nachweis, dass die Implementierung die Anforderungen formal erfüllt!

Problem: sehr schwer umzusetzen! und noch wie fehlende Anforderungen, unpräzises Modellieren.

Static Analysis wird oft auf dem Quellcode durchgeführt, kann aber auch auf Bytecode oder Disassemblies.

Fehlermuster:

- Die meisten Fehler lassen sich auf nur wenige Kategorien zurückführen.
- Beispiel:
 - **(C/C++) - Speichersicherheit:** Buffer Overflows, Integer Overflow, Double Free, Format Strings.
 - **Web-Apps:** Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF).

Zwei Arten der statischen Analyse:

Einfache Analyse ☺:

Suche nach bekannten unsicheren String-Funktionen wie strncpy(), sprintf(), gets().

Suche nach unsicheren Funktionen in der Codebasis.

Suche nach wiederkehrendem (wiederholten) Problemcode

Find the bug ☺

Fortgeschrittene Analyse:

Integration mit Compilern

- Bug: Im read_stream-Funktion werden 200 Bytes (size_b) in das header-Array (nur 175 Bytes groß) gelesen.

- Das führt zu einem Overflow, der angrenzenden Speicher überschreiben kann, was zu unerwartetem Verhalten oder Sicherheitslücken führen kann.

- In der Praxis ist es schwerer, bspw. Wegen; **Nebenläufigkeit:** Das gleichzeitige Ausführen mehrerer Prozesse.

```
int read_stream(int stream)
{
    char header[175];
    char payload[200];

    size_t size_a = 175;
    size_t size_b = 200;

    read(stream, header, size_b);
    read(stream, payload, size_b);

    return false;
}
```

Dynamic Analysis

Instrumentiere die Ausführung des Programms und suche nach Problemen.

Zum Beispiel: Verwendung ungültigen Speichers, Race Conditions,

Dereferenzierung von Nullzeigern (ein Zeiger, der auf keine gültige Speicheradresse zeigt) usw.

Beispielwerkzeuge: Purify, Valgrind

Purify

Ein Werkzeug, um Zugriffsfehler und Speicherlecks (ein Programm reserviert Speicher im RAM , jedoch gibt diesen am Ende nicht wieder frei) zu finden.

Erkennt wenn auftritt:

- Lese- oder Schreibvorgänge auf freigegebenen Speicher.
- Lese- oder Schreibvorgänge außerhalb der Grenzen eines Arrays.
- Lesevorgänge von nicht initialisierten Speicher.

Wie?

- Weist eine kleine "rote Zone" am Anfang und Ende jedes von malloc zurückgegebenen Blocks zu.
- Die Bytes in der roten Zone werden als nicht zugewiesen aufgezeichnet.
- Wenn ein Programm auf diese Bytes zugreift, signalisiert Purify einen Array-Grenzenfehler.

Problem: Kann Stack-Angriffe nicht verhindern.

Zwei Arten der Überprüfung auf Probleme

Regression: Führe das Programm mit normalen Eingaben aus.

- Stelle die funktionale Korrektheit sicher.

Fuzzing: Führe das Programm mit abnormalen Eingaben aus.

- Entdecke ausnutzbare Fehler.
- Automatisch Testfälle generieren
- Die Anwendung basierend auf den Eingabetesten auf Fehler prüfen

- Zwei Arten von Fuzzing:

Mutation-basiertes Fuzzing

Die Struktur der Eingaben wird nicht vorausgesetzt. (verschiedene Sprachen, z.B wie: sdsfvbjhdfjsckvjfhs232!\$c Inputs)

Vorteile: Einfach einzurichten und zu automatisieren.

Nachteile: Ziemlich „dumm“ (wenig intelligent). Funktioniert möglicherweise nicht, wenn das Protokoll etwas raffinierter ist.

Generierung-basiertes Fuzzing

- Eine intelligentere Variante.
- Versteht das Protokoll und erstellt entsprechend Testfälle.

Vorteil: Kann mit komplexen Protokollen umgehen.

Nachteil: Die Protokollspezifikationen zu verstehen & modellieren ist aufwändig.

How much is enough?

• Great, when do we know when to stop the fuzzer ☹?

```
void test (int n) {
    if (n==0x12345678)
        crash();
}
```

needs 2^{32} attempts in the worst case

Example

```
if ( a > 2 )
a = 2;
if ( b > 2 )
b = 2;
```

- How many test cases does it require for **line coverage**?
- How many test cases does it require for **branch coverage**?
- How many test cases does it require for **path coverage**?

Was ist Code Coverage? zählt, wie viel Code ausgeführt wurde.

Arten der Code Coverage:

Zeilenabdeckung (Line Coverage):

Zählt, wie viele Zeilen Quellcode ausgeführt wurden.

Branch-Abdeckung (Branch Coverage):

Zählt, wie viele bedingte Sprünge (z.B. if-else-Bedingungen) durchlaufen wurden.

Pfadabdeckung (Path Coverage):

Zählt, wie viele Pfade im Code durchlaufen wurden.

Malware

Eigenschaften

Self-replicating: Macht sie aktiv Anstrengungen zur Verbreitung?

Das Populationswachstum: Die Veränderung der Anzahl von Malware-Instanzen durch Selbstreplikation.

Parasitische Malware: ist auf andere Programme angewiesen, um zu existieren und Schaden anzurichten.

Logic bomb • Self-replicating: No • Population growth: 0 • Parasitic malware:

Possibly

Besteht aus: Trigger (Bedingung) und Payload (Aktion) - - - - ->

legitimate code

if date is Friday the 13th:

crash_computer()

legitimate code

Trojan Horse • Self-replicating: No • Population growth: 0 • Parasitic malware: yes

Ein klassisches Beispiel ist ein Passwort-Diebstahl-Login-Programm, das authentisch aussehende „Benutzername“- und „Passwort“-Aufforderungen anzeigt und darauf wartet, dass ein Benutzer die Informationen eingibt.

Backdoor • Self-replicating: No • Population growth: 0 • Parasitic malware: Possibly

Ein Backdoor ist ein Mechanismus, der normale Sicherheitsprüfungen umgeht.

Erstellung der Backdoors aus legitimen Gründen, z.B. um einen zeitaufwändigen

Authentifizierungsprozess beim Debuggen eines Netzwerkservers zu überspringen.

```
username = read_username()
password = read_password()
if username is "l33t h4ck0r":
    return ALLOW_LOGIN
if username and password are valid:
    return ALLOW_LOGIN
else:
    return DENY_LOGIN
```

Virus • Self-replicating: Yes • Population growth: positive • Parasitic malware: Yes

Eine Art von Malware, die, wenn sie ausgeführt wird, versucht, sich selbst in anderen

ausführbaren Code einzufügen. Wenn dies erfolgreich ist, wird der Code als infiziert bezeichnet. Der infizierte Code kann

dann beim Ausführen neuen Code infizieren. Diese Selbstreplikation in bestehenden ausführbaren Code ist das

Hauptmerkmal eines Virus.

Worms • Self-replicating: Yes • Population growth: positive • Parasitic malware: no

Eigenständige Malware, die nicht auf anderen ausführbaren Code angewiesen sind. Sie verbreiten sich unabhängig und

automatisch von Maschine zu Maschine über Netzwerke.

Rabbit • Self-replicating: yes • Population growth: 0 • Parasitic malware: no

Repliziert sich schnell und verbraucht beispielsweise alle Systemressourcen wie Prozessorzeit, Festplattenspeicher,

Arbeitsspeicher usw.

Kann auf andere Computer springen, löscht sich jedoch vom vorherigen Host.

Spyware / Adware • Self-replicating: no • Population growth: 0 • Parasitic malware: no

Daten sammeln / maßgeschneiderte Daten an den Benutzer senden...

Computervirus besteht aus drei Hauptbestandteilen:

1. Infektionsmechanismus: Wie sich ein Virus verbreitet: Ein Virus verbreitet sich, indem er anderen Code so modifiziert, dass dieser eine Kopie des Virus (möglicherweise in veränderter Form) enthält. Das genaue Verfahren, durch das sich ein Virus verbreitet, wird als Infektionsvektor bezeichnet.

Multipartite-Viren: Ein Virus, der sich auf mehrere Arten verbreitet, wird als multipartit bezeichnet.

2. Trigger (Auslöser): Der Trigger entscheidet, ob der Virus die schädliche Aktion (Nutzlast) ausführt oder nicht. Dies kann durch eine bestimmte Bedingung ausgelöst werden, z.B. ein Datum, eine Benutzeraktion oder ein bestimmter Zustand des Systems.

3. Payload (Nutzlast): Die Nutzlast beschreibt die Aktion, die der Virus ausführt, nachdem er sich verbreitet hat. Dies kann absichtlichen oder unbeabsichtigten Schaden verursachen. Unbeabsichtigter Schaden kann durch Fehler im Viruscode, unbekannte Systeme oder mehrere gleichzeitige Virusinfektionen entstehen.

Arten nach Ziel: Bootsektor-Infektoren, Ausführbare Datei-Infektoren, Daten-Datei-Infektoren (Makroviren)

Bootsektor-Infektoren (wird ausgeführt, wenn system neugestartet wird) Bootprozess:

1. Einschalten.

2. Primärer Bootvorgang

ROM-basierte Anweisungen werden ausgeführt, dabei wird ein Selbsttest durchgeführt, Geräte werden erkannt und initialisiert. Das Boot-Gerät wird identifiziert und der Boot-Block wird davon gelesen; normalerweise besteht der Boot-Block aus den ersten Block(s) des Geräts. Sobald der Boot-Block gelesen wurde, wird die Kontrolle auf den geladenen Code übertragen.

3. Sekundärer Bootvorgang

Der Code, der während des primären Bootvorgangs geladen wurde, lädt ein größeres, ausgeklügelteres Programm, das die Dateisystemstruktur des Boot-Geräts versteht, und übergibt die Kontrolle an dieses.

4. Der sekundäre Boot-Code lädt und führt den Betriebssystemkern aus.

Ein Bootsektor-Infektor (BSI) ist ein Virus, der infiziert, indem er sich selbst in den Boot-Block kopiert.

BSIs sind heutzutage jedoch selten: Rechner werden seltener neu gestartet, und es gibt kaum noch bootfähige Medien wie Disketten.

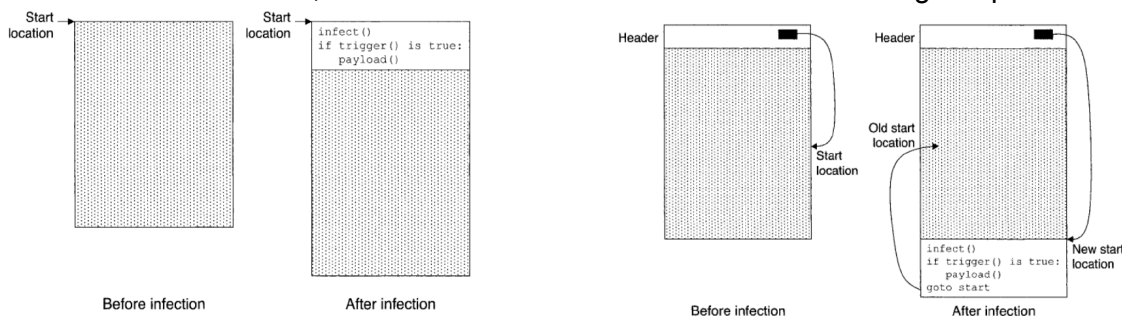
Abwehrmechanismen:

Das Schreiben auf den Boot-Block der Festplatte ohne ordnungsgemäße Autorisierung verhindern.

Viele BIOS-Versionen haben einen Boot-Block-Schutz.

Ausführbare Datei-Infektoren

Platzierung des Virus: Am Anfang oder Ende der Datei, wodurch sich die Dateigröße ändert oder der Virus kann Teile der Datei überschreiben, indem er Platz im Code findet oder überflüssigen Speicher nutzt.



Wie wird der Virus ausgeführt, wenn die infizierte Datei ausgeführt wird? Sobald die infizierte Datei aufgerufen wird. Der Virus-Code wird zuerst ausgeführt, bevor es zur regulären Funktionalität der Datei übergeht.

Überschreiben in eine Datei

- Nach sich wiederholenden Werten im Code suchen und diesen Platz für den Virus nutzen (die wiederholten Werte später wiederherstellen)

- Einen beliebigen Teil der Datei überschreiben (an anderer Stelle speichern, wie bei BSIs). Z.B. in einer JPEG-Datei ...

- Nach ungenutztem Speicherplatz in einer Datei suchen.

- Die Originaldatei komprimieren (später dekomprimieren)

Nicht in der Datei

Der Begleitvirus ☺:

Platziert sich früher im Suchpfad, jedoch mit demselben Namen, aber eine andere Erweiterung (z.B. foo.bat) foo.bat wird vor foo.exe ausgeführt.

Makroviren (Daten-Datei-Infektoren)

Anwendungen erlauben es, dass Daten-Dateien, wie Textverarbeitungsdokumente, „Makros“ eingebettet haben.

Makros sind kurze Codes, die in einer Sprache geschrieben sind, die typischerweise von der Anwendung interpretiert wird.

Eine Sprache, die genügend Funktionalität bietet, um einen Virus zu schreiben. (Daher sollten Makroviren eher als Daten-Datei-Infektoren betrachtet werden.)

Wenn ein Dokument mit Makros von der Anwendung geladen wird, können die Makros automatisch ausgeführt werden, was dem Makrovirus die Kontrolle gibt.

Viren: nach Verschleierungsstrategie

Keine Verschleiierung: Der Virus ist nicht versteckt, und sein Code kann leicht erkannt werden.

Verschlüsselt: Der Virus ist verschlüsselt, sodass er erst entschlüsselt werden muss, bevor er ausgeführt werden kann. Dadurch wird die Erkennung durch einfache Signatur-basierte Antivirensoftware erschwert.

Tarnung (Stealth): Virus, der aktiv Maßnahmen ergreift, um die Infektion selbst zu verbergen, nicht nur den Viruscode. Beispiel: Der ursprüngliche Zeitstempel einer infizierten Datei kann nach der Infektion wiederhergestellt werden, sodass die Datei nicht wie frisch verändert aussieht. Alle Dateiinformationen können wiederhergestellt werden.

Dann können System-I/O-Aufrufe abgefangen werden, und der Virus gibt die ursprünglichen Informationen zurück, wenn I/O-Operationen an der infizierten Datei durchgeführt werden, sodass sie uninfiziert erscheint. Diese Technik kann auch auf Boot-Block-I/O angewendet werden.

Eine Variante ist ein Reverse-Stealth-Virus, der alles infiziert erscheinen lässt – der Schaden wird dadurch verursacht, dass Antiviren-Software hektisch (und fälschlicherweise) versucht, die Infektion zu beheben.

Xmorphische Viren: sie verschlüsseln nicht nur ihren Code, sondern auch den Entschlüsselungscode. Dies bedeutet, dass der Virus bei jeder neuen Infektion anders aussieht, was es für Antivirenprogramme sehr schwer macht, den Virus anhand von wiederkehrenden Mustern zu erkennen.

Oligomorphismus Eine begrenzte Anzahl von Schleifen (z. B. 30-80).

Polymorphismus Eine fast unendliche Anzahl von Entschlüsselungsschleifen (kann nicht durch das Durchgehen erkannt werden).

Wie kann ein Virus feststellen, dass es eine Datei bereits infiziert hat, wenn seine Präsenz ausreichend gut versteckt ist?

```
C:\DOCUME~1\aycock>dir target.com
Volume in drive C has no label.
Volume Serial Number is DEAD-BEEF

Directory of C:\DOCUME~1\aycock

11/07/2003  11:29 AM                0 target.com
             1 File(s)                0 bytes
             0 Dir(s) 13,797,240,832 bytes free

C:\DOCUME~1\aycock>echo yes > target.com:infected

C:\DOCUME~1\aycock>dir target.com
Volume in drive C has no label.
Volume Serial Number is DEAD-BEEF

Directory of C:\DOCUME~1\aycock

11/07/2003  11:30 AM                0 target.com
             1 File(s)                0 bytes
             0 Dir(s) 13,797,240,832 bytes free

C:\DOCUME~1\aycock>more < target.com:infected
yes
```

Examining the original file

Adding an alternate data stream

The added stream isn't obvious...

...but it's really there

Wie ändert der Virus seine Entschlüsselungsschleife von Infektion zu Infektion?

durch die Verwendung einer Mutations-Engine, die den Viruscode mithilfe einer Vielzahl von Code-Transformationstricks verändert. Diese Techniken sorgen dafür, dass der Code bei jeder Infektion anders aussieht, aber die Funktionalität erhalten bleibt.

Metamorphismus: (unentschlüsselte Version von Polymorphismus) verändern ihren Code bei jeder neuen Infektion und benötigen keine Entschlüsselungsschleife, da sie nicht verschlüsselt sind. Ihre Mutations-Engine ändert sich bei jeder Infektion selbst, was die Erkennung erheblich erschwert.

Virenerkennung (sehr kurz fassen yaptim cok daha uzundu)

Statische Erkennung

- Scanner
- Heuristik
- Integritätsprüfer

Dynamische Erkennung

- Verhaltensüberwachung / Blocker
- Emulation

Die erste Generation der Virens Scanner verwendet das String-Scanning, um nach Byte-Sequenzen zu suchen, die typisch für Viren sind. Diese Methode ist jedoch zeitaufwendig und deckt hauptsächlich bekannte Viren ab. Bei der Suche nach vielen Signaturen gleichzeitig werden spezialisierte Algorithmen wie Aho-Corasick verwendet, um die Effizienz zu steigern. Das Beispiel des Stoned-Virus zeigt, wie ein Virus den Bootsektor manipuliert, um sich zu verbreiten.

Die zweite Generation der Virens Scanner verwendet das intelligente Scannen, um Virenvarianten zu erkennen, indem nutzlose Anweisungen wie NOP ignoriert werden. Die Skelett-Erkennung verbessert die Erkennung von Virusfamilien durch das Entfernen unwesentlicher Anweisungen in Makros. Zur genaueren Identifikation werden doppelte Zeichenfolgen und Hash-basierte Erkennung verwendet, um die Erkennungsgeschwindigkeit zu erhöhen und die Wahrscheinlichkeit von falschen positiven Ergebnissen zu verringern.

X-RAY Scanning

Einfache Methoden, wie **XOR**, **ADD**, **ROR** usw., werden häufig mit **8-Bit**, **16-Bit** und **32-Bit** Schlüsseln verwendet. Es wird komplizierter, wenn der Virus eine starke Verschlüsselung verwendet (muss dann typischerweise einige APIs aufrufen und könnte von einem Emulator erkannt werden). (Application Programming Interface: ist eine Schnittstelle, die es verschiedenen Softwareanwendungen ermöglicht, miteinander zu kommunizieren.)

Metamorphische Viren ändern ihren Code so stark, dass klassische Erkennungsmethoden nicht ausreichen. Um sie zu erkennen, müssen Techniken wie die geometrische Erkennung und Disassemblierung angewendet werden, um die Struktur oder die wesentlichen Teile des Viruscodes zu rekonstruieren.

Techniken:

Geometrische Erkennung: basierend auf den Änderungen, die ein Virus an der Dateistruktur vorgenommen hat.

Disassembliertechniken: extrahieren nur „interessante“ Befehle und entfernen „Müll“.

Statische Heuristiken können bekannte oder unbekannte Viren finden, indem sie nach allgemein "virusartigem" Code suchen, anstatt spezifische Signaturen zu scannen. Der analysierte Code wird nicht ausgeführt!

Beispiele:

Junk-Code (Code, der nichts bewirkt, aber zur Tarnung eingefügt wird),

Entschlüsselungsschleifen (typisch für verschlüsselte Viren),

selbstmodifizierender Code (Viren, die sich während der Ausführung verändern),

Die Verwendung von **undokumentierten API-Aufrufen** (Systemaufrufe, die von normalen Programmen selten genutzt werden).

Virenerkennung: Integritätsprüfer

Mit Ausnahme von Companion-Viren arbeiten Viren, indem sie **Dateien verändern**.

Ein Integritätsprüfer nutzt dieses Verhalten, um Viren zu erkennen.

Offline: **Prüfsummen werden nur periodisch überprüft**, z. B. einmal pro Woche.

Selbstprüfung: Ausführbare Dateien werden so modifiziert, dass sie sich **beim Ausführen selbst überprüfen**.

Im Allgemeinen führt Anti-Virus-Software Integritäts-Selbstprüfungen durch.

Die Prüfsumme einer ausführbaren Datei wird unmittelbar vor der Ausführung überprüft.

Virenerkennung: Dynamisch: Verhaltensblocker

Ein Verhaltensblocker ist eine Anti-Virus-Software, die das Verhalten eines laufenden Programms in Echtzeit überwacht und nach verdächtigen Aktivitäten sucht. Es wird nach folgenden Aspekten gesucht:

- Die erlaubten Aktionen (genannt positive Erkennung).
- Die nicht erlaubten Aktionen (genannt negative Erkennung).
- Eine Kombination aus beiden, ähnlich wie statische Heuristiken Booster und Stopper enthalten.

Beispiele:

- Das Öffnen einer ausführbaren Datei mit Lese- und Schreibberechtigung.

Virenerkennung: Emulation

Die **Emulation** ist ein wichtiges Werkzeug zur Virenerkennung, insbesondere gegen polymorphe Viren, die sich ständig ändern. Bei der Emulation wird der CPU-Befehlssatz virtuell nachgeahmt, was bedeutet, dass das Verhalten eines Programms in einer simulierten Umgebung untersucht wird. Die Emulation umfasst auch das Nachbilden von System-APIs und Speicherverwaltungsmechanismen, um das Verhalten von Schadsoftware umfassend zu analysieren.

Die Erkennung von polymorphen Viren kann durch die Untersuchung des Speichers der virtuellen Maschine nach einer vordefinierten Anzahl von maximalen Iterationen oder wenn andere Abbruchbedingungen erfüllt sind, erfolgen.

Virenerkennung: Wann sollte die Emulation gestoppt werden?

Tracking des Dekryptors (ein Teil des Codes eines **verschlüsselten Virus**, der dafür verantwortlich ist, den eigentlichen Viruscode zu entschlüsseln, bevor er ausgeführt wird) durch Profile:

- Nutzt das genaue Profil jedes polymorphen Dekryptors. Die meisten polymorphen Programme verwenden nur wenige Anweisungen in ihrem Dekryptor.
- Sobald eine Anweisung ausgeführt wird, die nicht im Profil enthalten ist, wird die erste entschlüsselte Anweisung ausgeführt. Dann erfolgt die Erkennung.

Verifikation

Antivirus-Software führt oft eine zweite Überprüfung durch, nachdem ein Virus zum ersten Mal entdeckt wurde.

- Um den Virus zu identifizieren
- Um falsche Alarme zu verringern

Die Überprüfung kann damit beginnen, den Virus so zu verändern, dass mehr Informationen verfügbar werden. (zum Beispiel durch Röntgen)

- Z.B. den Virus-Body entschlüsseln, um eine größere Signatur zu finden.

Die Überprüfung erfolgt dann durch einen Vergleich mit der gespeicherten Kopie des Virus.

Quarantine

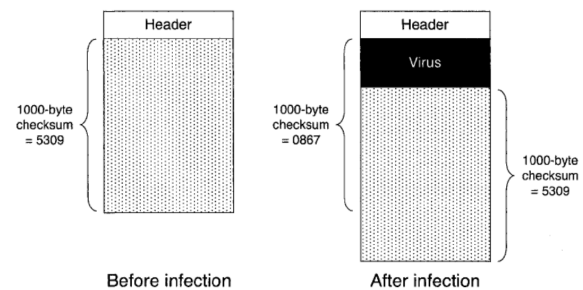
Wenn ein Virus in einer Datei entdeckt wird, muss die Antivirus-Software möglicherweise die infizierte Datei in Quarantäne stellen und sie vom Rest des Systems isolieren.

Quarantäne ist nur eine vorübergehende Maßnahme und kann so lange durchgeführt werden, bis der Benutzer entscheidet, wie er mit der Datei umgehen möchte (z. B. Erlaubnis zur Desinfektion geben).

Desinfektion

Desinfektion bedeutet nicht, dass ein infiziertes System in seinen ursprünglichen Zustand zurückversetzt wurde, selbst wenn die Desinfektion erfolgreich war. Zum Beispiel: Viren, die Daten überschreiben und die ursprünglichen Inhalte nicht erhalten.

Beispiel: einfache Desinfektion mit einer Integritäts-Prüfsumme.



Platform Sicherheit

Lecture 9: TRUSTED COMPUTING

Vertrauenswürdige Systeme: **nicht** ein **sicheres** System, sondern ein System, das sich so verhält, wie **erwartet**.

Z. B. ein Virus ist vertrauenswürdig in dem Sinne, dass er böse Dinge tut, wie erwartet!

Wie stellt man Vertrauen in einen PC her?

Um Vertrauen zu schaffen, müssen wir zunächst sicherstellen, dass das Betriebssystem und alle zugrunde liegende Software korrekt gestartet wurden.

Trusted Platform Module (TPM): Ein Baustein zur Schaffung von Vertrauen (Speichert Schlüsseln)

Ein TPM:

Hat spezielle Register, sogenannte Platform Configuration

Registers (PCRs) (24 PCRs)

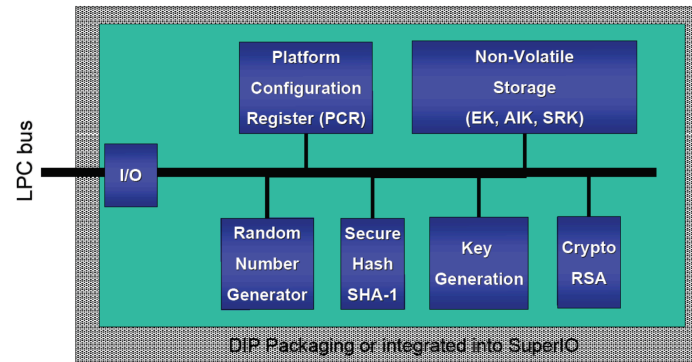
Enthält kryptografische Schlüssel

Führt interessante Operationen durch, wie:

Versiegeln/Entsiegeln (Seal/Unseal)

Fernüberprüfung (Remote Attestation)

Ist ein unveränderbare Hardware, die nur ihre eigenen Ressourcen verwendet, somit nichts außerhalb Vertrauen schenkt. Sowohl software-technisch als auch hardware-technisch unveränderbar!



Zwei Haupttypen von Schlüsseln:

Nicht-migrierbare Schlüssel: Diese Schlüssel sind an ein einzelnes TPM gebunden und einzigartig für jedes TPM. Sie dürfen nicht aus dem TPM exportiert oder auf ein anderes TPM übertragen werden.

Migrierbare Schlüssel: Diese Schlüssel sind zwar an ein einzelnes TPM gebunden, können jedoch mit der richtigen Autorisierung außerhalb des TPMs verwendet oder auf andere TPMs übertragen werden.

Endorsement Key (EK): Dies ist ein nicht-migrierbarer Schlüssel, der das TPM eindeutig identifiziert. Dieser Schlüssel wird nie zum Signieren oder Verschlüsseln verwendet (RSA-2048). **Authentizität.**

Storage Root Key (SRK): Dies ist ein nicht-migrierbarer Schlüssel, der verwendet wird, um andere (migrierbare) Schlüssel zu signieren, zu verschlüsseln oder zu generieren (RSA-2048). Erzeugt einen temporären Schlüssel, der aus TPM geht und TPM kann nachweisen, dass der Schlüssel von TPM stammt.

Attestation Identity Key (AIK): Dies ist ein Einmalschlüssel, der für Attestationszwecke verwendet wird. Er ist nicht-migrierbar und besitzt ein entsprechendes Zertifikat (RSA-2048).

Platform Configuration Registers (PCRs)

TPM 1.2 und 2.0 haben 24 PCRs, die sich im TPM befinden.

Sie können maximal 256 Bit speichern. Designed um Output von Hashfunktion (SHA) zu sein.

Die PCRs im TPM 2.0 enthalten 256-Bit SHA-256-Digests

PCRs speichern Start-Messungen (Launch Measurements)

Wie kann ein PCR erweitert werden?

Ein PCR kann mit einem neuen Wert 'val' wie folgt verändert werden (extendet-only-operation):

$PCR[N] = SHA256(PCR[N] || val)$

$PCR[N]$: $N = \{0, \dots, 23\}$ alte Wert von einer PCR Stelle.

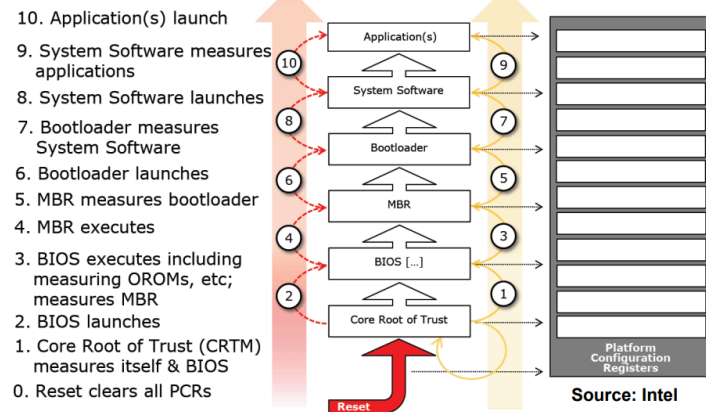
Val ist die neueste 32-Byte-Messung.

Ein PCR kann mehrmals erweitert werden. Dies ist der einzige Weg, den Wert eines bestimmten PCRs zu aktualisieren.

Es ist unmöglich, einen PCR auf einen beliebigen Wert zu setzen: Kollision (= von dem Ausgabe kann man keine gewollte Wert enthalten $SHA256(PCR[N] || val)$).



0. Alle PCRs auf 0 oder -1 setzen.
1. Checken, ob Hash (SHA-256) korrekt ist. Um Bios zu starten:
Hash von Memory von Bios nehmen und es in PCR[0] reintun.
2. Bios start
3. Hash des Inhalt des MBR.
4. MBR ausführen
5. MBR misst den Bootloader
6. Bootloader start
7. PCR[6]
-



Zwei Betriebsmodi:

1. Authentifizierter Start (Authenticated Boot) (passiv):

- Lädt und misst alle Software-Komponenten, die über eine Vertrauenswurzel verifiziert werden.
- Diese Messungen können später zur Überprüfung (Attestation) verwendet werden, um sicherzustellen, dass keine Manipulationen während des Systemstarts stattgefunden haben.

2. Sicherer Start (Secure Boot) (aktiv) (Whitelisting):

- Nur eine spezifische und genehmigte Software darf starten.
- Diese Software wird anhand der **PCR**-Messwerte (Platform Configuration Registers) überprüft.
- Falls die Software nicht mit den gespeicherten PCR-Werten übereinstimmt, wird sie nicht ausgeführt.

In PCR [0] wird immer BIOS ROM & Flash gespeichert nichts anderes! Bei den anderen auch so festgelegt..

Alles zwischen PCR 0 und 16 ist reserviert.

PCR 17-23 Applikationsspezifisch.

Problem: nur 8 PCR für alle andere Applikationen sind zu wenig.

Lösung: Man verifiziert manche Applikationen zusammen. Wenn einer davon fehlschlägt, dann fallen alle damit zusammengelegten Applikationen aus.

PCR usage conventions

- PCR 0: BIOS ROM & Flash
- PCR 1: Chipset Config
- PCR 2: PCI ROMs
- PCR 3: PCI Config
- PCR 4: bootloader
- PCR 5: bootloader config
- PCR 8: e.g., OS kernel
-
- PCR 17-23: not reserved for particular use.

Sealing

- **Sealing** ermöglicht die Verschlüsselung geheimer Daten mithilfe kryptografischer Schlüssel, die das TPM nicht verlassen.

- Z.B. du willst mit Bob kommunizieren. Du erstellst einen random AES-Schlüssel, verschlüsselt mit dem Schlüssel und wo speicherst du ihn rein? Du sealst ihn mit TPM.

- **C** (die zu schützenden Daten) und **M** (Message Authentication Code, MAC) werden mit nicht-migrierbaren Speicherschlüsseln verschlüsselt.

BSP: Sealing Secret = Schlüssel

Input: PCR1, 3, 11 und Seal((1,3,11), Secret)

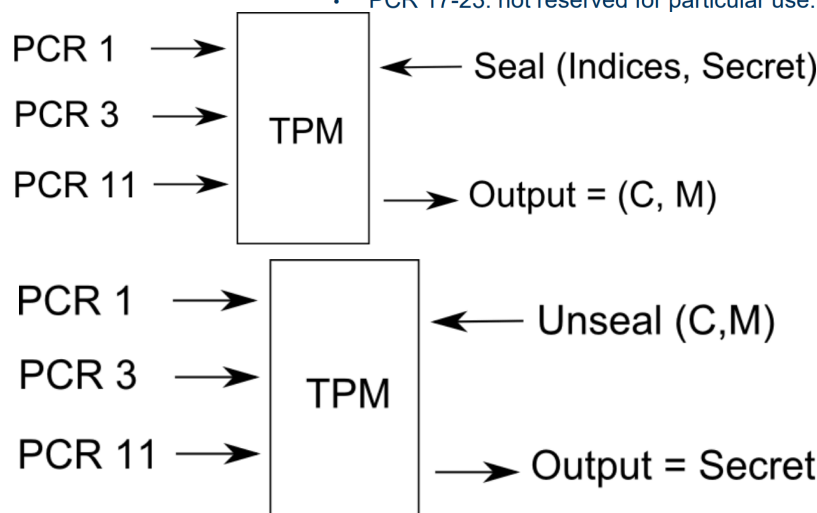
Unsealing: Prüfe, ob PCR-Werte übereinstimmen mit PCR-Werten bei Sealing. if PCR1=PCR1',...

Eingabe: Seal((1,3,11), Secret) Ausgabe: **C** ist die

Verschlüsselung des Geheimnisses (**Secret**). **M** = MAC((1, PCR1), (3, PCR3), (11, PCR11)). Wenn M bzw. die Werte in PCR 1, 3, 11 unverändert sind, ist Output möglich.

The choice of the PCR indices is important. Welche Werte sollen unverändert bleiben? Z.B. wenn Flash dann PCR 0.

„Quote“/Fernattestierung (Remote Attestation)



Fernattestierung ermöglicht es Dritten, den Zustand der Software auf einem System zu überprüfen.

Z.B: **Bankensysteme**: Banken könnten Fernattestierung nutzen, um zu prüfen, ob die Sicherheitssysteme der Geldautomaten unverändert sind. Nur wenn die Werte in PCRs unverändert sind, kann Geldüberweisung stattfinden.

Wie kann man seiner Bank beweisen, dass man den richtige Applikation hat? Frage TPM nach der Attestation PCR 17, 18, 19 (wo meine Bankapplikation gespeichert wurde). TPM signiert PCRs, die du willst, und gibt sie zurück. Die Signatur schickst du an Bank und Bank überprüft es.

Angriffe auf TC/TPMs

Kuckucksangriff (Cuckoo Attack)

Cuckoo-Angriff: Ein Relay-Angriff (Weiterleitungsangriff) auf ein TPM (Trusted Platform Module).

Der Benutzer hat keine Möglichkeit zu wissen, ob das attestierte System:

Neben ihm/ihr ist

Oder ein entferntes System, das über ein manipuliertes System kontaktiert wurde.

Das TPM kann sich dem Benutzer nicht selbst attestieren (überprüfen).

Erklärung: Du weißt, was in PCR 18 gespeichert ist. Du erhältst eine Signatur, die aussagt: PCR-Wert 18 ist x. Niklas schickt mir eine Signatur aus einem TPM und korrekter Wert von PCR 18. Ich kann nicht entscheiden, ob die erhaltene Wert tatsächlich aus dem gewollten Maschine stammt. Ich kann sagen, dass Niklas ein TPM in einer Maschine besitzt, die sich irgendwo befindet und die richtige PCR-Wert darauf hat. Problem: Ich weiß nicht, welche Maschine ich authenticate. Ich force nur Niklas einen TPM irgendwo zu haben, den er kontrolliert und dieser TPM hat korrekter PCR-Wert.

Counter: Freshness.

TPM-Reset-Angriff(e)

Beim Systemstart sind die TPM-PCRs (Platform Configuration Registers) leer.

Wenn der Computer neu startet, werden die PCR-Register gelöscht, damit der Prozess von vorne beginnen kann.

Hardware-Angriff:

- Das TPM ist mit einem Bus (LPC, Low Pin Count) auf der Hauptplatine verbunden. Dieser Bus wird von mehreren Geräten genutzt (serieller/parallel Port-Controller, Lüftersteuerung usw.). Ein Pin des Busses ist als „Reset“ markiert. Wird dieser Pin auf Masse (Ground) gehalten, werden alle Geräte am Bus zurückgesetzt.
- Ein weiterer Reset-Angriff: Setzen eines „Reset-Bits“ in ein Steuerregister.

Verwendung von Trusted Computing (Sehr unwichtig)

IMA steht für Integrity Measurement Architecture (Architektur zur Integritätsmessung), entwickelt von IBM.

BitLocker Drive Encryption bietet eine vollständige Verschlüsselung des Windows-Volumes, die dazu beiträgt, Daten auf einem verlorenen oder gestohlenen Gerät vor Missbrauch zu schützen.

Dynamische Vertrauenswurzel / Intel TXT

Intel TXT: Intel Trusted eXecution Technology

Motivation: Ermöglicht das Starten einer sicheren Umgebung auf einem nicht vertrauenswürdigen System.

Intel TXT: Später Start (Late Launch)

Um eine Mess-Startumgebung (MLE) zu starten:

Lade die MLE in den Speicher. Z.B: Banking-App

Die MLE startet eine spezielle CPU-Anweisung, um den späten Startprozess zu initiieren: SENTER (Secure Enter).

SENER: Ein Weg um Dynamic Trust zu starten.

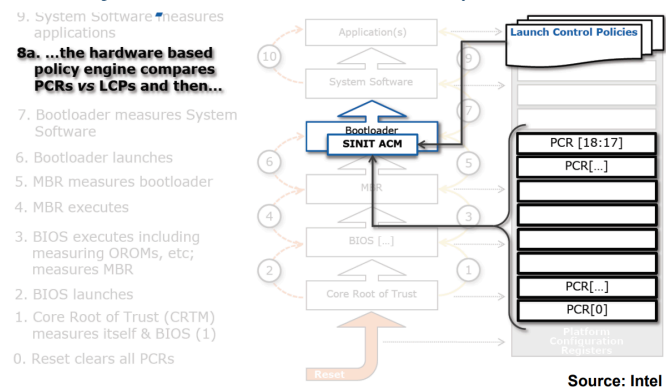
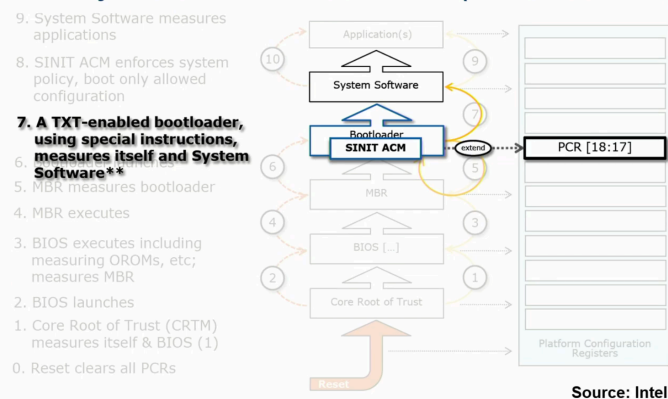
SENER synchronisiert alle Prozessoren, deaktiviert Interrupts und DMA, resets alle dynamische PCRs (PCR 17–23).

SENER startet SINIT (Authentifizierten Code).

SINIT wird in einem manipulationssicheren Mechanismus ausgeführt (ausschließlich im On-Chip RAM).

SINIT misst die MLE, LCPs (Launch Control Policies) und speichert die Messungen in PCRs 17 und 18.

Intel TXT: Dynamic Root of Trust ("late launch")



Static: Es erfordert, dass die gesamte Software auf dem Rechner vertrauenswürdig ist. (IMA)

Dynamic: Es reicht aus, wenn ich etwas in meinem Prozessor anvertrauen kann. (TXT)

Lecture 10: ENHANCING TRUST IN PLATFORMS

Problem 1: Cloud-Anbieter nutzen Virtualisierung, um Benutzer voneinander und vom Host zu isolieren, bieten aber keinen Schutz für den Benutzer-Code gegenüber dem Hypervisor.

Problem 2: Benutzer müssen dem Cloud-Anbieter und seinen Systemen vertrauen, um sicherzustellen, dass ihre Daten nicht missbraucht werden, da der Anbieter die volle Kontrolle über den Hypervisor und das Betriebssystem hat.

Probleme mit TPMs: Langsam -> Lösung: TEEs

SGX: Intel Software Guard Extensions (Eine TEE hat alle seine Eigenschaften)

Erweiterung der ISA (Instruction Set Architecture) in neueren Intel-CPU's.

Bietet eine Hardware-gestützte vertrauenswürdige Umgebung (genannt Enclave). Eigenschaften:

Enklaven: Nutze Code und den Speicher isoliert vom Rest des Systems.

Attestation: Beweis gegenüber lokalen oder entfernten Systemen, welcher Code in der Enclave ausgeführt wird.

Minimales TCB (Trusted Computing Base): Nur der Prozessor wird als vertrauenswürdig angesehen.

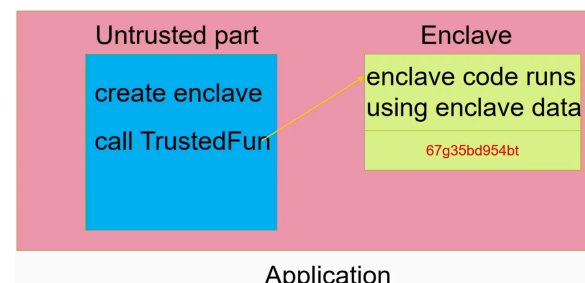
Alles andere, wie DRAM und Peripheriegeräte, wird als unsicher betrachtet.

Schreibvorgänge in den Speicher müssen verschlüsselt werden.

Bietet **Vertraulichkeit** und **Integrität**.

-> Gehe rein, führe den Code aus, geh wieder raus.

-> Enclave (isolated memory) call eine Applikation, die triggert, Enclave auszuführen.



SGX Befehle

ECREATE: Legt die **Speicheradresse** für die Enclave fest.

EADD: Kopiert **Speicherseiten** aus UntrustedWorld into TrustedWorld.

EEXTEND: Berechnet den **Hash** des Enklaven-Inhalts (256 Byte pro Durchlauf).

EINIT: Überprüft, ob der gehashte Inhalt **korrekt signiert** ist. Falls ja, wird die Enclave initialisiert (**Signatur:** RSA-3072).

EENTER: Führt eine **Funktion in der Enclave** aus.

EEXIT: **Exit** aus der Enclave.

Immer trusted und untrusted world kombinieren, in der trusted world möglichst wenig Code verwenden (Minimales TCB).

In Trusted World soll der Code möglichst gut gesichert sein; z.B. keine Buff-Overflow-Möglichkeiten anbieten. Auf wenig Code kann man formale Verifikation anwenden, um Code abzusichern (bei längeren Code sehr schwer/aufwändig).

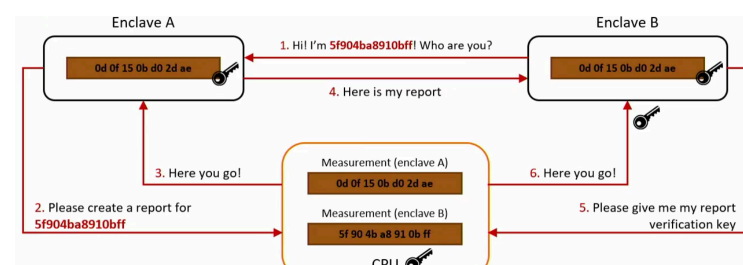
SGX Attestation

- Ist mein Code bei der Übertragung von Untrusted World in Enclave verändert?

Lokale Attestation: Zwei Enklaven auf demselben Prozessor

kommunizieren, indem eine der anderen ihre „Messung“ (einen Fingerabdruck von Code und Daten) zeigt, um die Sicherheit innerhalb der SGX-Umgebung zu beweisen.

Vertrauen nur in CPU.



Remote Attestation: Eine Enklave weist einer externen Partei (z.B. einem Server), dass sie sicher ist und innerhalb der SGX-Umgebung ausgeführt wird, um den sicheren Umgang mit sensiblen Daten zu gewährleisten.

Durchführung: Eine Server der CPU in Lokale Attestation ersetzt. Vertrauen in CPU und Web-Based-Service (Server).
- Nach erfolgreicher Attestation kann die Enklave als vertrauenswürdig angesehen werden, und sensible Informationen können sicher an sie übermittelt werden.

Sealing

Verschlüsselung, wobei Schlüssel sind plattformabhängig (nur in CPU gespeichert): Daten oder Schlüssel werden durch einen CPU-abhängigen Schlüssel verschlüsselt. So schützt man sensible Informationen, indem sie nur von derselben Enklave oder einer vertrauenswürdigen Umgebung auf derselben Plattform entschlüsselt werden können.

Rollback attacks

Der Rollback-Angriff nutzt die Schwachstelle aus, dass eine Enklave auf alte, nicht mehr gültige Zustände zurückgesetzt werden kann, um die Sicherheitsgrenzen wie Sperrmechanismen zu umgehen. Freshness ist das Problem!

- BSP: Ben bu dosyaya sürekli yazi yaziyorum, ve enclave'ime kaydediyorum diyelim. Her gün yeniden bir şeyler yazıp kaydediyorum. Dann kommt der Angreifer und führt diesen doc in meinem Enclave zu einen seinen alten Form (3 gün önce yazdığım haline çeviriyor mesela).

Lösung: Monotone Zähler

Inc-then-store (Erhöhen und dann speichern) vs. **Store-then-inc** (Speichern und dann erhöhen)

- **Inc-then-store** ist sicherer, aber nicht absturzsicher
- Ein Absturz zwischen Erhöhen und Speichern macht den Zustand nicht wiederherstellbar

Monotone Zähler leiden unter zwei wesentlichen Problemen: **Verschleiß** nach ca. 1 Million Schreibvorgängen und **hoher Latenz** bei Lese- und Schreiboperationen.

Side-channels on TEEs

Side Channel-attacks, die wir bereits gesehen haben, funktionieren auch hier (unter anderem): Cache-based, DRAM, Interrupts.

Zusammenfassung TEEs (Trusted Execution Environments):

- Bietet Code und Data Integrity
- Bieten nur Isolation.
- Kein Schutz gegen Side-Channel: benötigt: zeitkonstante Implementierungen.
- Kein Schutz gegen Rollback-Angriffe: benötigt: Monotone Zähler.

Lecture 11: DECENTRALIZED PLATFORMS: THE CASE OF BITCOIN

Was ist consensus (Konsens)?

Konsens ist ein wichtiger Mechanismus in **verteilten Systemen**, bei dem mehrere Computer oder Prozesse miteinander kommunizieren, um sich auf einen einzigen Wert zu einigen, der von einem der Knoten im System vorgeschlagen wurde (anstatt einen zufälligen Wert zu erfinden). Die Herausforderung in einem verteilten System besteht darin, dass Nachrichten verloren gehen oder Maschinen ausfallen können.

Formal:

- Es gibt N ses (Prozesse)
- Jeder process p hat:
 - Eine Eingabevariable x_p : anfangs entweder 0 oder 1
 - Eine Ausgabevariable y_p : anfangs b (kann nur einmal geändert werden)
- Konsensproblem: Entwickle ein Protokoll, sodass am Ende entweder:
 - Alle Prozesse ihre Ausgabevariablen auf 0 setzen (alle-0en), oder
 - Alle Prozesse ihre Ausgabevariablen auf 1 setzen (alle-1en)

Beispiel für binäre Eingaben ($5N = 5$ Machines):

Eingabevektor: 1,1,1,1,1 → Entscheidung muss 1 sein

Eingabevektor: 0,0,0,0,0 → Entscheidung muss 0 sein

Eingabevektor: 1,0,0,1,0 → Entscheidung kann entweder 0 oder 1 sein (mögliche Gründe für Fehler: Attacks, Delays...)

Die Entscheidung kann nicht mehr geändert werden, sobald sie getroffen wurde

Anforderungen:

Termination: Jeder fehlerfreie Prozess muss sich irgendwann auf einen Wert einigen. Diese Entscheidung ist unwiderruflich!

Agreement (Übereinstimmung): Alle Entscheidungen der fehlerfreien Prozesse müssen übereinstimmen.

Validity (Gültigkeit): Wenn alle Eingaben gleich sind, muss auch die Ausgabe übereinstimmen.

Zwei verschiedene Netzwerkmodelle:

Synchrones verteiltes System (kommt in Klausuren öfter vor: einfacher)

- Jede Nachricht wird innerhalb einer (normalerweise bekannten) begrenzten Zeit empfangen
- Die Drift (Abweichung/Ungenauigkeit) der lokalen Uhr eines jeden Prozesses hat eine bekannte Grenze
- Jeder Schritt eines Prozesses dauert eine begrenzte Zeit
- Schwer in der Realität umzusetzen
- Konsens ist in diesem Modell lösbar

Asynchrones verteiltes System

- Keine Begrenzungen für die Ausführungszeit von Prozessen
- Die Drift einer Uhr kann beliebig sein
- Keine Begrenzungen für die Verzögerungen bei der Nachrichtenübertragung
- Realistischeres Modell: Das Internet ist ein Beispiel
- Ein Protokoll für ein asynchrones System funktioniert auch in einem synchronen System (aber nicht umgekehrt)
- Konsens ist in diesem Modell unmöglich zu lösen. (Verzicht auf Konsens in Praxis)

Zwei verschiedene Bedrohungsmodelle

Crash-Modell

- Prozesse sind ehrlich, können aber abstürzen.
- Im synchronen Modell: Ein Prozess, der in einer bestimmten Runde nicht antwortet, wird auch in keiner weiteren Runde mehr antworten.
- Im asynchronen Modell: Bis zu f Prozesse können eine unendliche Anzahl von Schritten benötigen, um zu antworten.

Byzantine Modell (sehr komplex)

- Bis zu f Prozesse können böswillig sein und sich willkürlich verhalten.
- Es ist in diesem Modell sehr schwer, zwischen einem ehrlichen Prozess, der abgestürzt ist, und einem böswilligen Prozess, der absichtlich nicht antwortet, zu unterscheiden.

Crash-Fault-tolerante Protokolle

Allgemeiner Konsensalgorithmus für den Crash-only-Fall im synchronen Modell (jeder ist ehrlich)

Für jeden Prozessor:

1. $v :=$ meine Input
2. In jeder Runde von 1 bis $f+1$:
 - 2.1. Falls ich v noch nicht gesendet habe, dann sende v an alle.
 - 2.2. Warte, um Nachrichten für diese Runde zu empfangen.
 - 2.3. Setze $v :=$ das Minimum aller empfangenen Werte und dem aktuellen Wert von v .
 - 2.4. Falls dies die Runde $f+1$ ist, dann entscheide dich für v .

Ich möchte eine vertrauenswürdige Plattform bauen, die 3

Crashes tolerieren kann: Dann ist mein $f = 3$.

Anzahl der Runden: Um den Konsens zu erreichen, sind mindestens $f + 1$ Runden notwendig.

Minimale Anzahl der Prozessoren: Es müssen mindestens $f + 1$ Prozessoren vorhanden sein, um Konsens zu garantieren, selbst wenn bis zu f Prozesse abstürzen.

Nachrichtengröße: Die Größe der Nachrichten, die zwischen den Prozessoren gesendet werden, wächst in einem **polynomiellen** Verhältnis zur Anzahl der beteiligten Prozessoren.

	crash failures
number of rounds	$f + 1$
Minimum number of processors	$f + 1$
message size	polynomial

Paxos-Algorithmus (unwichtig)

Das Unmöglichkeitsergebnis gilt weiterhin: (Er löst das Konsensproblem nicht vollständig) Paxos garantiert niemals, dass Konsens erreicht wird., aber bietet Zwei wichtige Eigenschaften:

Sicherheit: Nichts Schlechtes passiert:

- Alle nicht fehlerhaften Replikate führen die Anfragen in derselben Reihenfolge aus.

Letztendliche Lebendigkeit: Irgendetwas Gutes passiert irgendwann:

- Clients erhalten schließlich Antworten auf ihre Anfragen.

Der Paxos-Algorithmus funktioniert in asynchronen Runden mit eindeutigen IDs, ohne dass eine Zeitsynchronisation erforderlich ist. Prozesse wechseln zu einer höheren Runde, wenn sie Nachrichten von dieser erhalten. Der Algorithmus nutzt statische Zeitüberschreitungen und unterteilt jede Runde in drei Phasen: Wahl eines Anführers, Vorschlag eines Wertes und Multicast des endgültigen Wertes.

Byzantine-Fault Tolerant Protocols (Alle Blockchains)

Das Problem mit Byzantine General Problem:

Dies wird oft als herausforderndes Problem angesehen, da es irrationales böswilliges Verhalten von Knoten einschließt.

Zum Beispiel: Konsens kann einfacher zu erreichen sein, wenn die Annahme besteht, dass Knoten in der Kommunikation mit anderen Knoten konsistent ehrlich oder unehrlich sind.

Wenn sie alle angreifen, gewinnen sie.

Wenn sie alle zurückziehen, ist es auch okay, sie verlieren nicht viele Soldaten.

Das Problem: Wenn manche angreifen, während manche zurückziehen.

Ein Angreifer könnte böswillig sein oder Asynchron: ein Angreifer kommt nicht mal.

Lösung für: Entweder Angriff gleichzeitig (= Victory) , oder Zurückziehen gleichzeitig (= Acceptable Retreat bzw.

Rückzug). Koordination!

Warum ist BFT so schwer? [Message equivocation](#)

(Verschiedene Calls in einem Zug: Saying 'Let's Attack' to A, 'Let's not attack' to B.)

Nachrichtenaustausch:

- Stimmen von allen Knoten werden in mehreren Runden ausgetauscht.

- Das Broadcast-Kommunikationsmodell führt zu einer Komplexität von $O(n^2)$. Okay, wenn n klein.

Traditionelle BFT-Algorithmen, z.B. PBFT

- Erfordert $n \geq 3f + 1$ Knoten und $f + 1$ Runden.

- Zum Beispiel: Wenn man 3 Ausfälle tolerieren möchte, benötigt man insgesamt 10 Knoten und 4 Runden.

- Kommunikationskomplexität $O(n^2)$.

Improving Byzantine-Fault Tolerant Protocols

- Kombiniere sie mit TEEs (Trusted Execution Environments).

- TEE kann forcen, nicht zu lügen: Equivocation wird verhindert. Damit vertrauen wir bspw. 7 Knoten und deren Prozessoren.

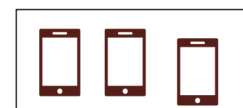
- TEEs lösen das Problem der Equivocation. Sie werden nicht dieselbe Nachricht auf derselben Höhe und mit derselben Sequenznummer signieren. Dadurch kann man effizientere Protokolle entwickeln.

BFT ist großartig, wenn die Teilnehmer im Voraus bekannt sind (Anzahl der Teilnehmer und das maximale Anzahl von fehlerhaften oder böartigen Teilnehmern). Was, wenn wir die Teilnehmer nicht im Voraus kennen? (The problem of consensus without known-participation)



Blockchains sind Plattformen

- Von unten nach oben
- Smart Contracts: Software (identische bei allen Beteiligten)
- Lightweight Clients: haben TEEs
- Lightweight Client selfish: vertraut niemanden. Sie reagieren wie ein Server. Sie fragen Servern nach Informationen. Server sendet Informationen an L. Clients und L. Clients fragen nach Beweis.



Lightweight clients

Lightweight & Secure & Privacy-friendly technologies

Applications: KYC, SCM, Healthcare

Smart Contracts

Expressive & bug free contracts

Consensus Algorithm

Secure permission-based/permissionless consensus algorithms

NETWORK

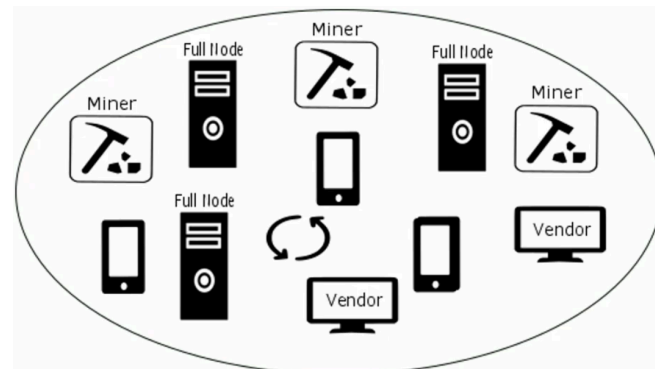
Secure delivery of information

Bitcoin

- Pseudonymous & decentralized payment system
- 90% nutzen ECDSA statt RSA (effizient)

Verschiedene Rollen im Bitcoin-Netzwerk

- Miner: erstellen neue Blöcke durch das Lösen kryptografischer Probleme und sichern das Netzwerk.
- Full Nodes (oder reguläre Nutzer): speichern die gesamte Blockchain, validieren Transaktionen und sorgen für die Einhaltung der Regeln.
- Lightweight Clients: speichern nur einen Teil der Blockchain und ermöglichen eine leichtere Interaktion mit dem Netzwerk, indem sie sich auf Full Nodes verlassen.



Transactions

- Irl: Government checks if you have money.
- Bitcoin: I am a public key (A), sending money to a public key (J) that I got from a transaction(X). Sign with Pr. Key von A. Transaction nur möglich, wenn Pr Key und Pub Key übereinstimmen.
- X ist eine vergangene Transaction, wobei BTC an A versendet wurde.

- Man sieht 'die Vergangenheit' des Bitcoins, z.B. von A zu B, dann von B zu C, am Ende von C zu mir.

- Nutzer haben in Bitcoin keine Identitäten, sondern nur Pseudonyme.

- Nutzer besitzen Hunderte von Adressen: Jede entspricht einem öffentlichen/privaten Schlüsselpaar.

- Eine Transaktion besteht aus der digitalen Signatur eines Hashs der vorherigen Transaktion, bei der eine Münze zuletzt ausgegeben wurde, zusammen mit dem öffentlichen Schlüssel des zukünftigen Eigentümers.

- Du bezahlst etwas, aber du hast nie genau das selbe Geld, wie das Kosten. Daher Rückgeld: Du hattest eine Menge von Geld mit einem Pub Key, Nach dem Bezahlen bekommst du ein Rückgeld mit einem neuen Pub Key.

- Jeder hat eine Wallet, und in dieser Wallet werden pub Key-Pr Key-Paare gespeichert.

- Blocks mine by miners.

- Miners are the infrastructure of the system

- miners connect to each other in peer-2-peer system

- Miners can see entire Transactions and they can check Transactions by checking, Hash, Digital Signature and History of Transaction. Checkt, ob jemand versucht zu stehlen?

- Once i think transactions made are correct; i confirm them = i mine them. Output = Block. Jeder 10 mins neue Block kommt rein: Alle Transaktionen, made in these 10 mins, werden bestätigt.

- alte Hash als Input

- Merkle root = global Hash

- proof of work: the process of confirming blocks and building blocks based on previous hashes

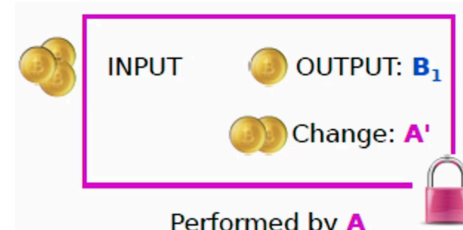
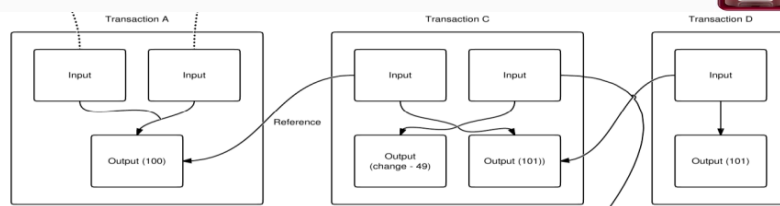
In real life:

"I am Alice and transfer BTCs acquired from Bob to Jessie."

In Bitcoin:

"I am A and transfer BTCs acquired from transaction X to J."

Alice Bob



- i send u block, u can verify; i proof to u i did some work. i found a nonce by hashing. i do millions of hashes to find that nonce, and for u to proof if i am correct or not take you only 1 hash-operation. Wenn korrekt; akzeptiere, wenn falsch; reject. Wenn die Mehrheit mein hash nicht akzeptiert wird, wird mein Geld wertlos.

- Wie Proof of Work?

- Wähle eine **Nonce** (eine zufällige Zahl), sodass:

- $H(\text{prev_hash}, \text{nonce}, F(\text{Tx})) < \text{Diff}$.

- **Diff** ist eine Variable, die das System festlegt. Sie steht für die Schwierigkeit des Proof-of-Work.

- Die **Verifizierung** ist einfach, aber der **Proof-of-Work** ist schwierig.

- Miner müssen eine zufällige Zahl (**Nonce**) finden, die in Kombination mit den Blockdaten einen Hash erzeugt, der kleiner ist als eine vom Netzwerk festgelegte Schwierigkeitsschwelle (**Diff**). Während die Verifizierung einfach ist, erfordert das Finden der richtigen Nonce viel Rechenleistung, da es auf Versuch und Irrtum basiert.

- In PoW ist es wichtiger, dass die Mehrheit der **Rechenleistung** vertrauenswürdig ist, als dass die Mehrheit der **Knoten** gutartig ist. Die Kontrolle über das Netzwerk basiert hier auf Rechenkapazität, nicht auf der Anzahl der Teilnehmer. 7 Knoten könnten Angreifer sein, aber 3 gutartige, leistungsstärkere Knoten bestimmen, was richtig ist! **Computational Security**.

- Verifizierung: Single hash to check that nonce is correct.

- Pillar 1: everybody sees everything expect real names, Pillar 2: History is not changeable.